

**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING**

Khwopa College of Engineering

Libali, Bhaktapur

Department of Computer and Electronics Engineering



**A FINAL REPORT ON
Nepali Sign Language Recognition for Emergency Phrases**

*Submitted in partial fulfillment of the requirements for the
degree in Bachelor of Computer Engineering*

Submitted by

Asul Limbu	KCE080BCT005
Dipesh Singh	KCE080BCT008
Manee Das Shrestha	KCE080BCT013
Sajan Sahikarmi	KCE080BCT035

Under the Supervision of

Er. Prakash Chandra Prasad

Khwopa College of Engineering

Libali, Bhaktapur

August, 2026

CERTIFICATE OF APPROVAL

This is to certify that this minor project work entitled “**Nepali Sign Language Recognition for Emergency Phrases**”, submitted by Asul Limbu (KCE080BCT005), Dipesh Singh (KCE080BCT008), Manee Das Shrestha (KCE080BCT013) and Sajan Sahikarmi (KCE080BCT035) has been examined and accepted as the partial fulfillment of the requirements for the degree of Bachelor in Computer Engineering. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

.....
Er. Surendra KC
External Examiner
Director of Engineering
Infinite Software Services

.....
Er. Prakash Chandra Prasad
Project Supervisor
Assistant Professor
IoE, Pulchowk Campus

.....
Er. Dinesh Man Gothe
Head of Department
Department of Computer and Electronics Engineering
Khwopa College of Engineering

COPYRIGHT©

The author has agreed that the library of Khwopa College of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the project work recorded herein or, in their absence, by the Head of the Department where the project report was completed. It is understood that recognition will be given to the author of the report and to the Department of Computer and Electronics Engineering, KhCE, for any use of the material in this project report. Copying, publication, or any other use of this report for financial gain without the approval of the department and the author's written permission is prohibited. Requests for permission to copy or make any other use of the material in this report, in whole or in part, should be addressed to:

Head of Department

Department of Computer and Electronics Engineering
Khwopa College of Engineering (KhCE)
Libali-08, Bhaktapur, Nepal

ACKNOWLEDGEMENT

We are grateful to our project supervisor, **Er. Prakash Chandra Prasad**, for his guidance throughout this project. His questions about how we intended to evaluate the system, asked early and repeatedly, are the reason this report reports signer-independent accuracy instead of a number that would have looked better and meant less.

We thank our Head of Department, **Er. Dinesh Man Gothe**, for his encouragement and for the departmental support that made the work possible.

This project could not have been built without Nepal's Deaf community. We thank **Yojana Sakha** and her colleagues at the **Rehabilitation Centre for the Rural Deaf (RCRD)** in Bhaktapur, and the resource persons at the **National Federation of the Deaf Nepal (NFDN)** in Kathmandu, who gave us their time, taught us the correct signed form of every phrase in this report, and recorded reference clips for us. Any part of this system that works does so because they were willing to teach us first.

We also thank the Department of Computer and Electronics Engineering, Khwopa College of Engineering, for the resources and the environment in which this work was carried out, and our friends and families for their support.

Asul Limbu	KCE080BCT005
Dipesh Singh	KCE080BCT008
Manee Das Shrestha	KCE080BCT013
Sajan Sahikarmi	KCE080BCT035

ABSTRACT

Nepali Sign Language (NSL) is the first language of Nepal’s deaf community, yet very few hearing people understand it. The gap is most dangerous in an emergency, when a deaf person needs to convey an urgent message immediately and no interpreter is present. This project built a system that recognises six NSL emergency phrases from an ordinary camera and delivers each one as on-screen text and speech.

No public dataset of NSL emergency phrases exists, so one was recorded from scratch. The natural signed form of every phrase was established in person with Deaf experts at the Rehabilitation Centre for the Rural Deaf and the National Federation of the Deaf Nepal before any recording began. Eight signers then contributed 840 clips across seven classes: the six emergency phrases and a negative class covering non-signing motion.

Each frame is reduced by MediaPipe to 225 landmark coordinates instead of being stored as video, which keeps the representation compact and independent of background and lighting. Landmarks are normalised against two anatomical anchors, the shoulder midpoint for the body and the wrist for each hand, and every clip is standardised to 146 frames, the 95th percentile of the recorded frame-count distribution. A bidirectional Long Short-Term Memory network classifies the resulting sequence.

Evaluation used leave-one-signer-out folds, in which the model is scored only on a signer absent from its training data. Accuracy averaged 95.51% over eight folds and 95.95% pooled across all 840 predictions, with per-class F1 between 0.950 and 0.992 for the six emergency phrases. The same architecture reaches 98.41% under a random split, and that gap of 2.5 percentage points is a measure of how much a random split overstates performance on this kind of data. A seventh, negative class trained on non-signing motion lets the system stay silent instead of announcing an emergency phrase for arbitrary motion. The trained network was exported to TensorFlow Lite and runs entirely on an Android phone, which extracts landmarks on the device, classifies the sign, displays the recognised phrase, and speaks it aloud without a network connection. A streaming inference server is retained as an optional fallback for devices that cannot sustain on-device landmark extraction.

Keywords: *Nepali Sign Language, Emergency Phrases, Sign Language Recognition, MediaPipe Holistic, Hand Landmarks, BiLSTM, Signer-Independent Evaluation.*

Contents

Copyright	i
Acknowledgement	ii
Abstract	iii
List of tables	viii
List of figures	ix
List of Symbols and Abbreviation	xi
1 INTRODUCTION	1
1.1 Background Introduction	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
2 LITERATURE REVIEW	4
2.1 Review of Related Works	4
2.1.1 NSL Dictionary (Acharya and Sharma, 2003)	4
2.1.2 ASL Character Recognition Using CNN (Masood et al., 2018)	4
2.1.3 Word-Level Sign Recognition Using CNN and RNN (Masood et al., 2018)	4
2.1.4 NSL Translation Using CNN (Mali et al., 2018)	5
2.1.5 MediaPipe Hands (Zhang et al., 2020)	5
2.1.6 Deep Learning for NSL Gesture Recognition (Ligal and Baral, 2022)	5
2.1.7 ISL Recognition Using MediaPipe Holistic (Goyal and Velmathi, 2023)	5
2.1.8 The NSL23 Dataset (Sunuwar et al., 2024)	5
2.1.9 NSL Letter Detection Using MediaPipe and CNN (Shrestha et al., 2024)	6
2.1.10 NSL Characters Recognition with Deep Learning (Poudel et al., 2025)	6
2.2 Review Matrix	6
2.3 Research Gap	7
2.4 Related Theories	8
2.4.1 Emergency Phrases as Dynamic, Multi-Sign Sequences	8
2.4.2 Landmark Extraction with MediaPipe Holistic	8

2.4.2.1	Why Pose Landmarks Are Included	9
2.4.2.2	Why Face Landmarks Are Excluded	9
2.4.2.3	Why Landmarks and Not Pixels	9
2.4.3	Dual-Anchor Landmark Normalisation	9
2.4.3.1	Hand Normalisation	9
2.4.3.2	Pose Normalisation	10
2.4.4	Sequence Standardisation	11
2.4.5	The LSTM Cell and the Bidirectional LSTM	11
2.4.6	Classification and Confidence Thresholding	12
2.4.7	Signer-Independent Evaluation	12
3	METHODOLOGY	14
3.1	Software Development Approach	14
3.2	Phase 0: Establishing the Signed Form of Each Phrase	15
3.2.1	The Negative Class	16
3.3	Phase 1: Dataset Collection	16
3.3.1	The Recorder Application	16
3.3.2	Recording Setup and Variation	17
3.3.3	Storing Landmarks Instead of Video	17
3.3.4	Aggregation Across Signers	17
3.4	Phase 2: Feature Extraction	18
3.5	Phase 3: Determining the Sequence Length	18
3.6	Phase 4: Normalisation and Standardisation	18
3.7	Phase 5: Model Training	19
3.7.1	Architecture	19
3.7.2	Why This Model and Not a Larger Pretrained One	20
3.7.3	Training Configuration	20
3.8	Phase 6: Evaluation Methodology	21
3.8.1	Leave-One-Signer-Out	21
3.8.2	The Random-Split Contrast	21
3.8.3	The Architecture Comparison	22
3.8.4	Metrics	22
3.9	Phase 7: Deployment	22
3.9.1	Revision of the Deployment Approach	22
3.9.2	On-Device Recognition	23
3.9.3	Model Export	24
3.9.4	The Fallback Inference Server	25

3.9.5	Frame Rate as a Correctness Constraint	25
3.9.6	Mobile Client	26
3.9.7	The Decision Rule	27
4	SYSTEM DESIGN	28
4.1	Requirement Specification	29
4.1.1	Software Requirements	29
4.1.2	Hardware Requirements	29
4.1.3	Functional Requirements	30
4.1.4	Non-Functional Requirements	31
4.2	Feasibility Assessment	31
4.2.1	Economic Feasibility	31
4.2.2	Technical Feasibility	32
4.2.3	Operational Feasibility	32
4.3	Use Case Diagram	32
4.4	System Block Diagram	33
4.5	Data Flow Diagrams	34
4.5.1	Level 0	34
4.5.2	Level 1	35
4.6	Sequence Diagram	35
4.7	Class Diagram	36
4.8	Activity Diagram	37
5	RESULT AND DISCUSSION	39
5.1	Experiment Setup	39
5.2	Dataset Composition	40
5.3	Sequence Length and Standardisation	40
5.4	Training Behaviour	41
5.5	Signer-Independent Recognition Results	43
5.5.1	Per-Class Performance	43
5.5.2	Confusion Matrix	44
5.6	What the Random Split Shows	46
5.7	Generalisation and Overfitting	47
5.8	Architecture Comparison	48
5.9	Why the Accuracy Is High	50
5.10	Model Export Verification	51
5.11	Deployment	52
5.12	Development Process	53

5.13	Behaviour as the Vocabulary Grows	55
5.13.1	Confusion Grows Faster Than the Vocabulary	55
5.13.2	The Data Requirement Scales With Classes, Not Total Clips	55
5.13.3	The Negative Class Gets Harder	56
5.13.4	The Confidence Threshold Needs Recalibrating	56
5.13.5	Predicted Shape of the Degradation	56
5.14	Discussion	57
5.14.1	What the Results Support	57
5.14.2	Eight Signers Is the Binding Limitation	57
5.14.3	The Negative Class Is the Weak Point	58
5.14.4	Duration Is Only Encoded Below the Sequence Length . . .	58
5.14.5	What Moving On-Device Cost	59
6	CONCLUSION	60
6.1	Conclusion	60
6.2	Limitations	61
6.3	Future Work	62
	REFERENCES	65
A	Field Consultation with NSL Experts	66
B	Data Collection Tool	68
C	Dataset Organisation	71
D	Sample Landmark Data	73
D.1	Feature Vector Layout	73
D.2	Raw Values	73
D.3	After Normalisation	74
D.4	After Length Standardisation	74
E	Client Interface	75
F	Server Interface	76

List of Tables

2.1	Review matrix of related works	7
3.1	Target emergency phrases and their categories	15
3.2	Classifier architecture	19
4.1	Software used, and the role of each component	29
5.1	Clips by class and signer in the final dataset	40
5.2	How the 840 clips were standardised to 146 frames	41
5.3	Leave-one-signer-out results. Each fold trains on seven signers and tests on the eighth.	43
5.4	Per-class performance, pooled over all eight folds	44
5.5	Pooled confusion matrix. Rows are the true class, columns the prediction.	45
5.6	The two evaluation protocols compared	46
5.7	The three levels of accuracy, and what the gap between each pair means	47
5.8	Architecture variants under the leave-one-signer-out protocol, ranked by mean accuracy. Units are given as first recurrent layer / second recurrent layer / dense head; a dash marks a layer the variant does not have.	48
5.9	TensorFlow Lite export verification	52
5.10	Measured costs on the fallback server path	53
5.11	On-device measurements, read from the client’s own telemetry on a physical device	53
5.12	Development increments	54
5.13	Predicted effect of a larger vocabulary on each component	56
D.1	Layout of the 225-value feature vector	73
D.2	Raw pose landmarks 0 to 4, first frame	73
D.3	Normalised pose landmarks 0 to 4, first frame	74
F.1	Server endpoints	76
F.2	Messages on the streaming endpoint	76

List of Figures

2.1	The two normalisation anchors on the MediaPipe Holistic skeletons. Filled points are ordinary tracked landmarks; the open circles and dashed line mark the anchor points and the reference distance each block is normalised against.	10
3.1	Agile incremental development approach	14
4.1	Use case diagram of the NSL emergency-pharse recognition system	33
4.2	System block diagram. The offline path builds the model; the phone reproduces the same preprocessing and runs the classifier; the server offers the same path over a network as a fallback. The two shaded boxes are the same preprocessing definition.	34
4.3	Level 0 data flow diagram	34
4.4	Level 1 data flow diagram	35
4.5	Sequence diagram for one on-device recognition	36
4.6	Class and module diagram. Solid arrows are use relationships; dashed arrows are imports or verification relationships. The heavier box on the right is the client’s port of the Python preprocessing contract, pinned to it by generated fixtures.	37
4.7	Activity diagram for one on-device recognition attempt	38
5.1	Distribution of raw clip frame counts across the 840 recorded clips, with the 95th-percentile cutoff that fixes the sequence length at 146 frames	41
5.2	Accuracy per epoch for each leave-one-signer-out fold. Training and validation curves are drawn from signers inside the training set; the dashed line marks the held-out signer’s score.	42
5.3	Loss per epoch for each fold. Validation loss is the quantity early stopping monitors.	42
5.4	Confusion matrix pooled over all eight leave-one-signer-out folds . .	45
5.5	Mean leave-one-signer-out accuracy for each architecture variant, with 95% confidence intervals over the eight folds. The vertical rule marks the shipped model’s mean; every interval crosses it. . . .	49
A.1	The project team with the NSL consultant following a gloss consultation session	66

A.2	Reviewing a recorded demonstration with the consultant to confirm the reference form of a phrase	66
A.3	An NSL expert demonstrating the signed form of a phrase for reference recording	67
B.1	A signer recording the <i>Call the police</i> phrase, showing live pose and hand landmark overlays alongside the signer identity and phrase selection	69
B.2	The recorder during an active recording, showing hand landmarks in detail and the running per-class clip counts on disk	70
C.1	The shared dataset repository, with raw and processed directories tracked separately and a commit history showing contributions merged from several signers	71
C.2	The raw directory after merging, with one subfolder per phrase class holding the contributions of all signers	71
C.3	Local directory layout: raw clips organised by phrase class, alongside the processed output directory	72
C.4	Contents of the processed directory: the compiled tensors alongside the manifest and the sequence-length metadata	72

List of Symbols and Abbreviation

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
ASL	American Sign Language
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DFD	Data Flow Diagram
F1	F1 Score, the harmonic mean of precision and recall
FPS	Frames Per Second
GPU	Graphics Processing Unit
HMAC	Hash-Based Message Authentication Code
HTTP	Hypertext Transfer Protocol
ISL	Indian Sign Language
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
LOSO	Leave-One-Signer-Out
LSTM	Long Short-Term Memory
MCP	Metacarpophalangeal (finger knuckle joint)
NFDN	National Federation of the Deaf Nepal
NSL	Nepali Sign Language
RCRD	Rehabilitation Centre for the Rural Deaf
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SDLC	Software Development Life Cycle
TFLite	TensorFlow Lite
TTS	Text to Speech
UI	User Interface
WS	WebSocket
WSS	WebSocket Secure
YUV	Luminance and Chrominance Colour Encoding

CHAPTER 1

INTRODUCTION

1.1 Background Introduction

Sign languages are complete natural languages with their own vocabulary and grammar. They are not signed versions of spoken languages, and they do not translate word for word into them. Nepali Sign Language (NSL) is the language of the deaf community in Nepal and differs substantially from American, British, or Indian sign languages. It grew within Nepal’s deaf community after the first deaf school opened in Kathmandu in the 1960s, and the standard gesture set is published by the National Federation of the Deaf Nepal [1, 2].

The community that uses it is larger than official figures suggest. The 2011 national census recorded roughly 80,000 deaf or hard-of-hearing Nepalis. The National Federation of the Deaf Nepal puts the real figure above 300,000 and attributes the difference to the undercounting of a condition that is not visible to an enumerator [3, 4]. For most of these people sign language is the primary means of communication, and almost none of the hearing population can understand it.

This gap is inconvenient in ordinary life and dangerous in an emergency. A deaf person who needs to report a fire, ask for an ambulance, or tell a bystander that they cannot breathe has no reliable way to do so if no interpreter is present, and interpreters are scarce and cannot be summoned on demand. The time spent failing to communicate is time that matters.

Recognition technology has advanced quickly for well-resourced sign languages, particularly American Sign Language, but NSL has received very little of that attention. Only one public NSL dataset exists and it covers the fingerspelling alphabet [2]. The handful of published NSL recognition systems target individual letters or static handshapes [5–7]. Nothing published addresses emergency communication, and nothing addresses whole phrases, which are what a person actually needs to sign under pressure.

Two developments make a lightweight system practical. Pretrained hand and body tracking such as MediaPipe [8] extracts the geometry of a signing person from an ordinary camera without gloves or depth sensors, and reduces each video frame to a few hundred numbers. Sequence models such as the bidirectional Long Short-Term Memory network [9, 10] classify the resulting motion directly. Together they allow a recognition system to be trained from a few hundred recordings and to run on hardware people already own.

This project applies that combination to a narrow, high-value problem: recognising six NSL emergency phrases from a live camera feed and delivering each as on-screen text and speech. The six phrases are *I can't breathe* (मलाई सास फेर्न गाह्रो भइरहेको छ), *The building is on fire* (भवनमा आगो लागेको छ), *Call the police* (प्रहरीलाई फोन गर्नुहोस्), *I need an ambulance* (मलाई एम्बुलेन्स चाहियो), *Help me / I am in danger* (मलाई मददत गर्नुहोस् / म खतरामा छु), and *I need to go to the toilet* (मलाई शौचालय जानु छ). They span medical, fire, crime, and basic-need situations. Their signed form was established with Deaf experts before any data was recorded, so what the system recognises is real NSL and not an invention of the authors.

1.2 Problem Statement

In an emergency, clear communication is the difference between help arriving and help not arriving. Deaf and hard-of-hearing people in Nepal are at a disadvantage in exactly those moments:

- Most hearing people, including first responders and bystanders, do not understand Nepali Sign Language.
- Human interpreters are scarce and are almost never present when an emergency begins.
- Existing sign language recognition systems are built for high-resource sign languages. No system or dataset exists for NSL emergency communication [2, 6, 7].

A deaf person in an emergency may therefore be unable to convey an urgent need to the people around them. What is missing is an accessible, low-cost tool that recognises essential NSL emergency phrases as they are signed and presents them in a form a hearing person understands immediately.

1.3 Objectives

The primary objective was to design and implement a system that recognises six Nepali Sign Language emergency phrases and outputs them as text and audio. The specific objectives were:

- To establish the natural signed form of each of the six target phrases with NSL experts, so that the recorded data reflects how the phrases are actually signed.

- To build a custom dataset of NSL emergency phrases across multiple signers and recording conditions, since no suitable public dataset exists.
- To develop a landmark-based recognition pipeline using MediaPipe Holistic, representing each frame as 225 pose and hand coordinates.
- To normalise landmarks against anatomical anchors and standardise every clip to a fixed length derived from the recorded data.
- To train a bidirectional LSTM classifier and evaluate it on signers absent from its training data.
- To deploy the trained model in a mobile application that runs recognition on the device itself, so that no network connection is required at the moment of an emergency, and that on the user's command captures a sign, displays the recognised phrase, speaks it aloud, and withholds output when the system is not confident.

1.4 Scope of the Project

The system recognises exactly seven classes: the six emergency phrases listed above and a seventh negative class covering non-signing motion, idle movement, and gestures outside the target set. Each input is one complete phrase performed once. Recognition works from an ordinary camera with no gloves, markers, or depth sensors, and the model runs without a GPU.

The seventh class deserves a note. A classifier trained on six phrases will assign one of those six to any input it receives, including someone scratching their head. For a system whose output is an emergency announcement, that behaviour is unacceptable. The negative class gives the model explicit examples of what is not a sign, so the confidence threshold described in Chapter 3 can withhold output for inputs unlike anything in the six target phrases.

Outside the scope are sentence-level or conversational translation, the full NSL vocabulary, recognition of more than one person at a time, and facial expression. Facial landmarks were excluded after the NSL consultants confirmed that none of the six target phrases depends on facial expression to be understood. The system is a proof of concept for a fixed phrase set and is built so that phrases, signers, and deployment targets can be added later.

CHAPTER 2

LITERATURE REVIEW

Sign language recognition sits at the intersection of computer vision and sequence modelling. Published work divides along two axes: whether the input is raw pixels or extracted landmarks, and whether the signs studied are static hand-shapes or motion. Emergency phrases are whole dynamic signs, so this project is concerned with the landmark-based and temporal branches of the field. This chapter reviews the works closest to that problem, summarises them, states the gap they leave, and then sets out the theory the system is built on.

2.1 Review of Related Works

2.1.1 NSL Dictionary (Acharya and Sharma, 2003)

Acharya and Sharma [1] compiled the *Nepali Sign Language Dictionary* for the National Federation of the Deaf Nepal. It remains the standard reference for the NSL gesture set. Being a printed reference it offers no recognition capability, and it does not cover the multi-sign emergency phrases this project targets, which is why the signed form of each phrase had to be established directly with Deaf experts.

2.1.2 ASL Character Recognition Using CNN (Masood et al., 2018)

Masood et al. [11] trained a convolutional neural network on raw images for American Sign Language character recognition and reported strong accuracy on a large image dataset. The work shows that convolutional networks handle static finger-spelling well when data is plentiful. It also illustrates how much data a pixel-based model needs, and it does not address motion.

2.1.3 Word-Level Sign Recognition Using CNN and RNN (Masood et al., 2018)

In a companion paper, Masood et al. [12] combined a convolutional network with a recurrent network to recognise word-level gestures from video. This established the pattern of pairing spatial feature extraction with temporal modelling for dynamic signs, which is the same principle applied here through a BiLSTM. The computational cost of running a convolutional network on every frame is the reason this project extracts landmarks instead.

2.1.4 NSL Translation Using CNN (Mali et al., 2018)

Mali et al. [5] built one of the earliest NSL recognition systems, a convolutional translator over a very small vocabulary. The signer was required to wear red gloves at test time, which limits practical use, and the system handled only static gestures.

2.1.5 MediaPipe Hands (Zhang et al., 2020)

Zhang et al. [8] introduced MediaPipe Hands, a pretrained pipeline that converts each video frame into 21 three-dimensional hand keypoints in real time on ordinary hardware. By discarding background, lighting, and skin appearance it makes compact and data-efficient gesture representations possible. MediaPipe performs no classification of its own; it is the enabling technology on which this project’s representation is built.

2.1.6 Deep Learning for NSL Gesture Recognition (Ligal and Baral, 2022)

Ligal and Baral [13] explored deep architectures for NSL gesture recognition, pairing a convolutional network with a recurrent network in one variant and with a Vision Transformer in another. The work confirms research interest in NSL. It follows the pixel-based formulation and does not target emergency communication or deployment on modest hardware.

2.1.7 ISL Recognition Using MediaPipe Holistic (Goyal and Velmathi, 2023)

Goyal and Velmathi [14] applied MediaPipe Holistic to Indian Sign Language and used LSTMs for dynamic signs and convolutional networks for static ones. This is the closest published work in method to the approach taken here and supports the landmark-plus-temporal-model direction. It targets Indian Sign Language, and it produces no audio output.

2.1.8 The NSL23 Dataset (Sunuwar et al., 2024)

Sunuwar et al. [2] released NSL23, the first public NSL dataset, containing 630 videos of the 49 alphabet characters performed by 14 volunteers. It is distributed as video because some NSL signs are defined by motion. NSL23 demonstrates that landmark-friendly NSL video can be collected from a modest number of signers, which is useful evidence for a project that had to record its own data. It covers the alphabet only.

2.1.9 NSL Letter Detection Using MediaPipe and CNN (Shrestha et al., 2024)

Shrestha et al. [6] present the closest prior landmark-based NSL system. MediaPipe landmarks are re-rendered as skeleton images and a convolutional network is trained on those images, with high reported accuracy. Two observations matter here. The work confirms that MediaPipe extracts usable NSL hand geometry. It also converts the numeric landmark geometry back into pixels instead of feeding the sequence to a temporal model, and its evaluation does not separate signers between training and test, so the reported accuracy does not establish how the system behaves on a person it has not seen.

2.1.10 NSL Characters Recognition with Deep Learning (Poudel et al., 2025)

Poudel et al. [7] introduced a 36-class static image dataset and reported roughly 90% accuracy with fine-tuned MobileNetV2 and ResNet50 networks. The work confirms continued activity in NSL recognition and the continued dominance of the static-image formulation, which leaves motion-based phrase signs unaddressed.

2.2 Review Matrix

Table 2.1 summarises the reviewed works against the goal of this project.

Table 2.1: Review matrix of related works

Author (Year)	Approach / Focus	Limitation for this project’s goal
Acharya & Sharma (2003) [1]	Printed dictionary of the standard NSL gesture set	Reference only; no recognition; no emergency phrases
Masood et al. (2018) [11]	CNN on raw images for ASL characters	Static only; large data requirement; not NSL
Masood et al. (2018) [12]	CNN and RNN on video for word-level gestures	Compute-heavy; not NSL; no audio output
Mali et al. (2018) [5]	CNN-based NSL translator	Very small vocabulary; red gloves required; static only
Zhang et al. (2020) [8]	MediaPipe Hands real-time landmark tracking	Tracking only; performs no sign classification
Ligal & Baral (2022) [13]	CNN+RNN and CNN+ViT for NSL gestures	Pixel-based and compute-heavy; not emergency-focused
Goyal & Velmathi (2023) [14]	MediaPipe Holistic with temporal models for ISL	Targets ISL; no audio output
Sunuwar et al. (2024) [2]	NSL23: first public NSL alphabet video dataset	Alphabet only; a dataset, not a system
Shrestha et al. (2024) [6]	MediaPipe landmarks rasterised to images; CNN	Static only; not signer-independent; no temporal model
Poudel et al. (2025) [7]	Static image dataset; MobileNetV2 / ResNet50	Static only; excludes motion-based phrase signs

2.3 Research Gap

The reviewed literature leaves four gaps that together define this project:

1. No public NSL dataset or recognition system exists for emergency phrases. All published NSL work targets the fingerspelling alphabet or isolated characters.
2. Published NSL recognition is almost entirely static and pixel-based. None feeds numeric landmark sequences to a temporal model to recognise motion-based signs.
3. NSL recognition results are reported without separating signers between training and test, so published accuracies do not establish how the systems behave on an unseen person.
4. No existing NSL system produces both text and spoken output, and none addresses what should happen when the input is not a known sign at all.

2.4 Related Theories

2.4.1 Emergency Phrases as Dynamic, Multi-Sign Sequences

Fingerspelling spells a word letter by letter using held handshapes. An emergency phrase is a different object: a complete communicative act performed as a continuous movement. A phrase such as *I can't breathe* or *The building is on fire* involves both hands moving through characteristic trajectories, shifts in posture, and transitions between component signs, all spread across many frames. No single still image contains the phrase. The information is in the sequence, which is why each phrase is recorded as a short clip, converted to a sequence of per-frame vectors, and classified as a whole.

The natural NSL form of a phrase is also not a word-for-word rendering of its English translation. *I can't breathe* is not signed as three sequential signs for “I”, “can't”, and “breathe”; it may be one or two signs. Establishing the natural form of each phrase with an NSL expert was therefore a prerequisite to recording anything, and the expert's performance became the reference that every signer reproduced.

2.4.2 Landmark Extraction with MediaPipe Holistic

The central design decision is to represent each frame as a compact vector of body and hand landmarks instead of raw pixels. MediaPipe Holistic [8, 15] is a pretrained pipeline that tracks several components in a single frame:

- Body pose: 33 landmarks covering the upper body, each with x, y, z coordinates, giving 99 numbers.
- Left hand: 21 landmarks at the wrist and finger joints, giving 63 numbers.
- Right hand: 21 landmarks in the same layout, giving 63 numbers.
- Face: 468 landmarks, excluded from this project.

The three components that are used are concatenated in a fixed order to produce the 225-number feature vector used throughout the project:

$$\mathbf{v} = \underbrace{[v_0, \dots, v_{98}]_{\text{pose (99)}}}_{\text{pose (99)}}, \underbrace{[v_{99}, \dots, v_{161}]_{\text{left hand (63)}}}_{\text{left hand (63)}}, \underbrace{[v_{162}, \dots, v_{224}]_{\text{right hand (63)}}}_{\text{right hand (63)}} \in \mathbb{R}^{225}.$$

When a hand or the pose is not detected in a frame, that block is filled with zeros, so every frame produces exactly 225 numbers regardless of what was visible.

2.4.2.1 Why Pose Landmarks Are Included

Several of the target phrases involve arm trajectories and upper-body posture that hand shape alone does not capture. A 21-point hand mesh would record the shape of the hand but not where the arm carried it, and a sweeping fire-reporting gesture would become difficult to separate from a distress call. Including the 33-point pose mesh gives the model the motion arc of the arms and torso.

2.4.2.2 Why Face Landmarks Are Excluded

Facial expression carries grammatical meaning in sign languages generally. Including all 468 face points would add roughly 1,400 numbers per frame and increase the feature vector more than sevenfold. Before the dataset was recorded, the NSL consultants were asked directly whether facial expression changes or adds meaning for any of the six target phrases. They confirmed it does not for this particular phrase set, so the face block was left out. This is a property of these six phrases and not a general claim about NSL.

2.4.2.3 Why Landmarks and Not Pixels

The landmark representation was chosen for three reasons. A model over 225 coordinates per frame can be learned from a few hundred clips, where a pixel-based network would need far more. Landmarks discard background, lighting, and appearance, which are the factors that cause a pixel-trained model to fail in a room it has not seen. The resulting model is small enough to train on a laptop and to run without a GPU.

2.4.3 Dual-Anchor Landmark Normalisation

Raw landmark coordinates depend on where the signer stands and how far they are from the camera. Neither changes the meaning of the phrase, so both are removed before the data reaches the model.

2.4.3.1 Hand Normalisation

Each hand’s 21 points are translated so the wrist (landmark 0) is the origin, then divided by the distance from the wrist to the base of the middle finger (landmark 9). This makes the hand representation invariant to its position in the frame and to its apparent size:

$$\hat{\mathbf{h}}_i = \frac{\mathbf{h}_i - \mathbf{h}_0}{\|\mathbf{h}_9 - \mathbf{h}_0\| + \varepsilon}, \quad i = 0, \dots, 20,$$

where ε is a small constant guarding against division by a near-zero scale.

2.4.3.2 Pose Normalisation

The 33 pose landmarks are translated so the midpoint of the left shoulder (landmark 11) and right shoulder (landmark 12) is the origin, then divided by the shoulder width:

$$\hat{\mathbf{p}}_i = \frac{\mathbf{p}_i - \frac{1}{2}(\mathbf{p}_{11} + \mathbf{p}_{12})}{\|\mathbf{p}_{11} - \mathbf{p}_{12}\| + \varepsilon}, \quad i = 0, \dots, 32.$$

Figure 2.1 shows both anchors on the MediaPipe Holistic skeletons they are computed from. Landmarks 11 and 12 and their midpoint define the pose anchor; landmarks 0 and 9 define the hand anchor and the wrist-to-middle-knuckle distance used to scale it.

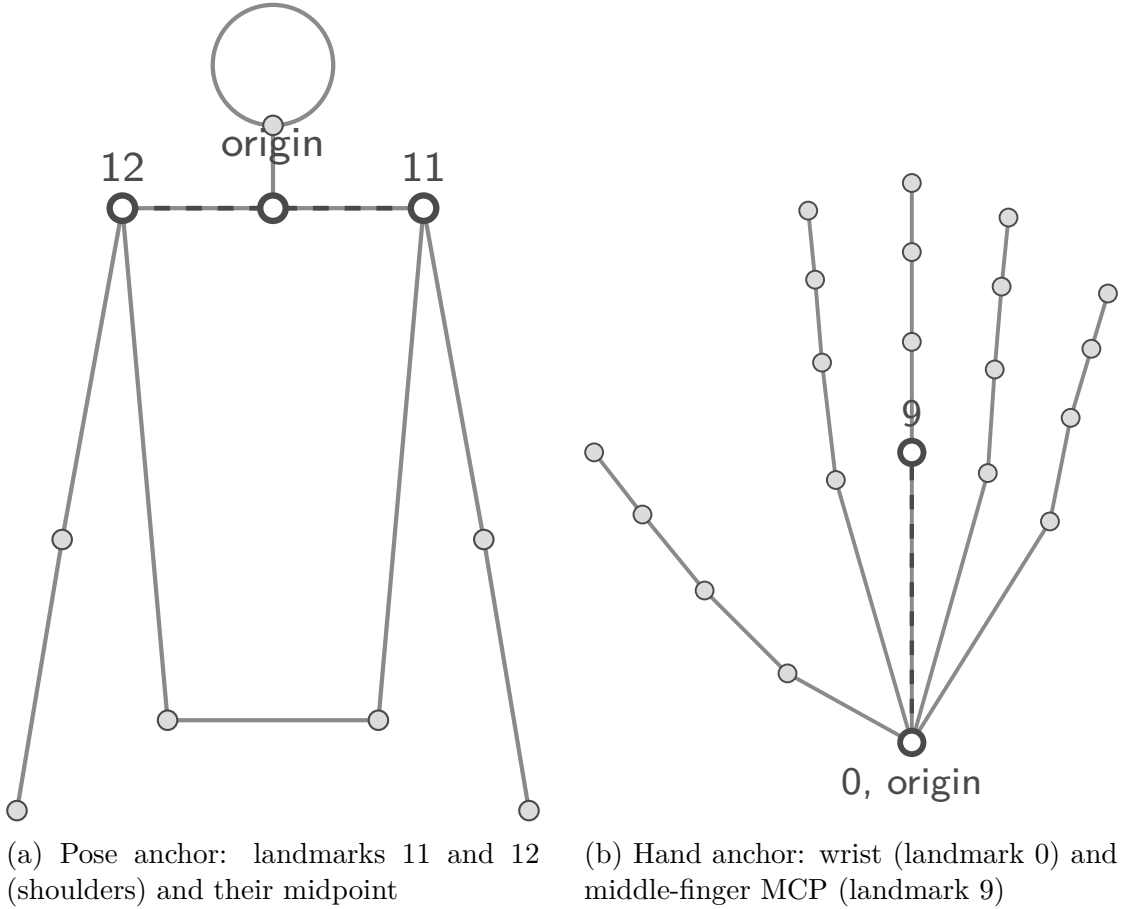


Figure 2.1: The two normalisation anchors on the MediaPipe Holistic skeletons. Filled points are ordinary tracked landmarks; the open circles and dashed line mark the anchor points and the reference distance each block is normalised against.

Two anchors are used because one cannot do both jobs. The shoulder anchor preserves where a hand sits relative to the body, which distinguishes a sign made at the chest from one made at the side. The wrist anchor captures finger configuration independently of where the hand is. Normalising the whole 225-vector against a

single anchor would lose one or the other.

Both blocks map a zero input to a zero output, which matters later: an undetected block and a padded frame are both exactly zero and can be treated identically.

2.4.4 Sequence Standardisation

A recurrent classifier needs every input to have the same number of time steps, but signers take different amounts of time to perform the same phrase. Every clip is therefore standardised to a fixed length, `seq_len`. The value is measured from the recorded data instead of assumed: the frame-count distribution across all clips is computed and the 95th percentile is taken.

- A clip longer than `seq_len` is subsampled at `seq_len` evenly spaced positions across its full duration, so the whole performance is still represented.
- A clip shorter than `seq_len` is zero-padded at the end, never looped, with a boolean mask recording which frames are real.

The 95th percentile is preferable to the maximum because a small number of unusually long clips would otherwise force every other clip to be padded out to their length, filling the tensor with padding that carries no signal.

2.4.5 The LSTM Cell and the Bidirectional LSTM

A Long Short-Term Memory network is a recurrent network designed to carry information across many time steps without the vanishing gradients of a plain recurrent network [9]. At each step t the cell keeps a cell state c_t and a hidden state h_t , and uses a forget gate f_t , an input gate i_t , and an output gate o_t to control what is discarded, added, and exposed:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (2.1)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad (2.2)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (2.3)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (2.4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (2.5)$$

$$h_t = o_t \odot \tanh(c_t), \quad (2.6)$$

where $x_t \in \mathbb{R}^{225}$ is the normalised landmark vector at step t , σ is the logistic sigmoid, $\odot$ is element-wise multiplication, and W , U , b are learned.

A standard LSTM reads in one direction, so its representation at step t depends only on the past. A bidirectional LSTM runs two such networks over the same sequence, one forward and one backward, and concatenates their hidden states [10]. This suits multi-sign phrases, where the end of a phrase often disambiguates its opening. Two phrases in this set begin with similar upward hand motion and separate only later; a forward-only model must commit to an interpretation of the opening before it has seen what follows.

2.4.6 Classification and Confidence Thresholding

The final hidden representation is passed to a fully connected layer with a softmax activation, converting raw scores z_k into a distribution over the $K = 7$ classes:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, 7. \quad (2.7)$$

The predicted class is $\hat{k} = \arg \max_k p_k$ and $\max_k p_k$ is the confidence. Training minimises the categorical cross-entropy against the true label.

At inference the system emits a phrase only when the confidence exceeds a threshold and stays silent otherwise. For a tool that announces emergencies, a wrong announcement is worse than no announcement, so the asymmetry is deliberate. A softmax over seven classes is also closed-world: it distributes probability across the classes it knows and has no direct way to express that an input resembles none of them [16]. This is the reason the negative class described in Section 3.2 is part of the model itself rather than a filter bolted on afterwards: giving the classifier explicit examples of non-signing motion during training is what lets the confidence threshold do useful work against inputs that are not a sign at all.

2.4.7 Signer-Independent Evaluation

How a recognition system is evaluated determines what its accuracy means. If clips are split into training and test sets at random, clips from the same person appear on both sides. The model can then reach a high test score by recognising individuals rather than signs, and the number says nothing about a new user. This is a form of data leakage [17], and it is the weakness identified in the closest prior NSL work [6].

The alternative used here is leave-one-signer-out evaluation. Every clip is tagged with the identity of the person who recorded it. For each signer in turn, the model is trained on all clips from the other signers and tested only on that signer’s clips. Every clip is scored exactly once, by a model that never saw its

signer. The mean across folds estimates accuracy on a new person, which is the quantity that matters for a tool intended for public use.

A signer can be held out only if every class is still present after their clips are removed, since a model cannot be fairly scored on a class it never saw.

CHAPTER 3

METHODOLOGY

This chapter describes how the system was built, in the order the work was carried out. The pipeline runs from establishing the signed form of each phrase with Deaf experts, through recording and preprocessing a custom dataset, to training the classifier, evaluating it, and deploying it. The evaluation protocol and the deployment path are described at the end, since both differ from what the proposal anticipated.

3.1 Software Development Approach

Agile incremental development was adopted because several parameters could not be fixed in advance. The sequence length depended on how long signers actually took to perform each phrase, the normalisation scheme had to be validated against real recordings, and the deployment target changed once the recognition results were in hand. Each increment delivered a working module: phrase verification, the recorder, the preprocessing core, the trained model, the inference server, the mobile client, and finally the on-device recognition path that replaced the server on the delivered system.

The riskiest unknown was addressed first. Before any large-scale recording, a small set of clips was captured to confirm that MediaPipe Holistic produces stable landmarks for multi-sign NSL phrases performed at natural speed. Had it not, the entire landmark-based approach would have needed reconsideration while there was still time to change it.

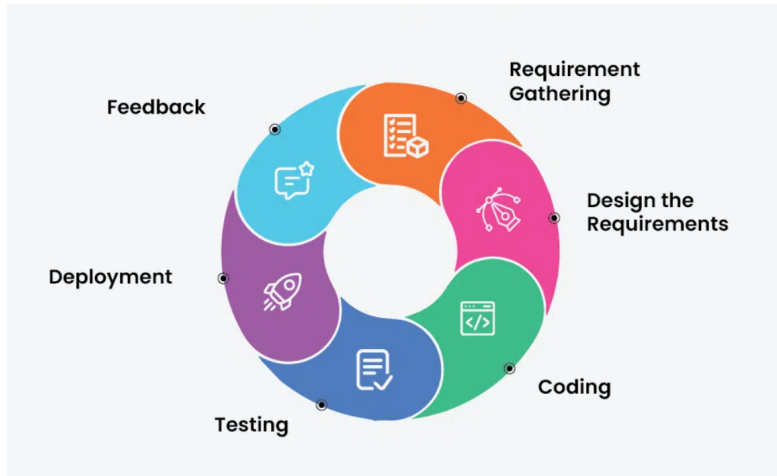


Figure 3.1: Agile incremental development approach

3.2 Phase 0: Establishing the Signed Form of Each Phrase

No standard published gloss exists for the six target emergency phrases, so the natural signed form of each had to be established with the Deaf community before recording could begin. Two consultations were held in person: with Yojana Sakha and colleagues at the Rehabilitation Centre for the Rural Deaf in Bhaktapur, and with a resource person at the National Federation of the Deaf Nepal in Ranibari Marga, Samakhushi, Kathmandu.

Each consultant was asked to demonstrate how the idea would be signed in an actual emergency, as concisely as possible, and not to sign the English sentence word by word. Each demonstration was discussed for regional variation, recorded as a reference, and confirmed as the form every signer would reproduce. Because the experts' time was limited, they also contributed roughly ten clips per phrase, which anchor the dataset to an authentic performance. Photographs from the consultations appear in Appendix A.

The consultants were also asked directly whether facial expression changes or adds meaning for any of the six phrases. They confirmed it does not for this phrase set, which settled the exclusion of the 468 face landmarks from the feature vector.

Table 3.1 lists the six phrases and the situation each covers. Phrase 6 replaced an earthquake phrase considered at proposal stage. The consultants advised that a request to use the toilet is a far more frequent communication failure for deaf people in institutional settings than an earthquake report, and the change was approved at the mid-term defence.

Table 3.1: Target emergency phrases and their categories

#	English phrase	Nepali phrase	Category	Class key
1	I can't breathe	मलाई सास फेर्न गाह्रो भइरहेको छ	Medical	cant_breathe
2	The building is on fire	भवनमा आगो लागेको छ	Fire	building_on_fire
3	Call the police	प्रहरीलाई फोन गर्नुहोस्	Crime	call_police
4	I need an ambulance	मलाई एम्बुलेन्स चाहियो	Medical	need_ambulance
5	Help me / I am in danger	मलाई मददत गर्नुहोस् / म खतरामा छु	Generic emergency	help_danger
6	I need to go to the toilet	मलाई शौचालय जानु छ	Basic need	need_toilet
7	Not one of the above	—	Negative class	none

3.2.1 The Negative Class

The seventh class was added during development and is not in the original proposal. A classifier over six phrases assigns one of those six to every input it receives, so a signer adjusting their glasses would be reported as an emergency. For a system whose output is a spoken emergency announcement, that is the worst available failure mode.

The **none** class holds deliberate non-examples: resting, idle movement, gestures outside the target set, and partial or abandoned signs. Giving the model explicit negatives lets it learn a decision boundary around the six real phrases instead of partitioning the whole input space among them, so the confidence threshold of Section 3.9.7 has a class to fall back on instead of being forced to choose one of six emergency phrases for every input.

3.3 Phase 1: Dataset Collection

Because no public dataset of NSL emergency phrases exists, one was recorded from scratch. This was the most consequential phase of the project: the ceiling on recognition accuracy is set by the quality and variety of the recordings.

3.3.1 The Recorder Application

A purpose-built recorder was written for the task. It runs MediaPipe Holistic live on a webcam feed and, for every frame, extracts a 225-value vector: 33 pose landmarks, 21 left-hand landmarks, and 21 right-hand landmarks, each with x , y , and z coordinates. The operator selects a signer identity and a phrase, then starts and stops each recording explicitly, so every clip corresponds to one deliberate performance of one phrase.

Every frame is mirrored horizontally before detection. This is recorded here because it is a constraint on everything downstream: the landmarks in the dataset describe a mirrored image, and any inference path that does not reproduce the same flip will present the model with the signer’s hands swapped. Screenshots of the recorder in use appear in Appendix B.

Frames in which a hand or the pose is not detected are zero-filled for that block and the detection rate is recorded. The frame is not dropped. Each clip is therefore accompanied by a JSON sidecar holding its frame count, duration, and per-block detection rates, which made it possible to identify weak recordings without replaying them.

3.3.2 Recording Setup and Variation

Variation was introduced deliberately so the model would not learn a single recording condition:

- Lighting: bright, dim, and mixed daylight near a window.
- Background: plain wall, cluttered room, and several different rooms.
- Distance: close to the camera and at arm’s length or further.
- Signers: eight people, differing in body size, hand size, and signing speed.
- Speed: each signer performed each phrase somewhat faster and somewhat slower across takes.

Each clip is saved as `<phrase>__<signer>__<index>.npy`. Encoding the signer in the filename is what makes signer-independent evaluation possible later, and it was the reason for adopting the convention.

3.3.3 Storing Landmarks Instead of Video

The recorder saves only the per-frame landmark vectors. Video is never written to disk. This keeps the dataset small, removes a slow reprocessing step from every later stage, and means no video of any signer is stored or transmitted.

The cost is worth stating plainly. Because no video was kept, the landmark representation cannot be recomputed. Changing the landmark source, for example to run detection on a phone instead of a server, cannot be evaluated against the existing data and would require re-recording every clip.

3.3.4 Aggregation Across Signers

The eight signers recorded independently on separate machines. Git was used to merge each contribution into one shared repository instead of copying files by hand, which gave a reviewable history of what each signer added and caught duplicate and misnamed clips before they entered the combined set. Screenshots of the merged repository and the resulting directory layout appear in Appendix C.

After aggregation the dataset held 840 clips: exactly 120 in each of the seven classes, contributed by eight signers. The composition is reported in full in Section 5.2.

3.4 Phase 2: Feature Extraction

Every frame of every clip is reduced to the same 225-dimensional vector, concatenated in a fixed order:

- 33 pose landmarks $\times (x, y, z) = 99$ values, at indices 0 to 98.
- 21 left-hand landmarks $\times (x, y, z) = 63$ values, at indices 99 to 161.
- 21 right-hand landmarks $\times (x, y, z) = 63$ values, at indices 162 to 224.

A clip is therefore a $(n_{\text{frames}}, 225)$ array of 32-bit floats, with n_{frames} varying between clips. This layout, the normalisation anchors, and the sequence length are held as constants in a single module that both the training code and the inference server import, so the two cannot drift apart.

3.5 Phase 3: Determining the Sequence Length

The classifier requires a fixed number of time steps, but raw clip lengths vary because signers take different amounts of time. The value was measured from the data rather than chosen.

Across the 840 clips, frame counts ranged from 30 to 217, with a median of 89 and a mean of 91.7. The 95th percentile of this distribution gives `seq_len = 146` frames. The distribution and the cutoff are shown in Figure 5.1 in Chapter 5. The value is written to a metadata file that every later stage reads, so no stage recomputes or assumes it. Nothing in the training code, the export, or the mobile client hardcodes it; the sequence length has moved twice as recordings were added, and each time the change propagated without a code edit.

An observation from the same metadata turned out to matter more than expected. The recorder never locked a frame rate; it ran as fast as the interface and MediaPipe allowed, giving a median capture rate of 15.7 frames per second across a range of 10.8 to 21.2. Since the model receives a fixed-length sequence with no explicit time axis, it reads frame count as duration. The capture rate is therefore a property of the training data that any real-time path has to reproduce, and Section 3.9.5 describes how.

3.6 Phase 4: Normalisation and Standardisation

Each clip is normalised and then standardised to `seq_len`, in that order.

Normalisation applies the dual-anchor scheme of Section 2.4.3 to each frame, treating the three landmark blocks independently. The pose block is re-centred on

the shoulder midpoint and divided by shoulder width; each hand block is re-centred on its wrist and divided by the wrist-to-middle-knuckle distance. Normalising per block keeps the position of a hand relative to the body while removing the signer’s position and distance from the camera.

Standardisation then forces the clip to exactly 146 frames in one of three ways. A clip already at 146 frames is used unchanged. A longer clip is subsampled at 146 evenly spaced positions spanning its full duration, so the whole performance is represented instead of the first 146 frames. A shorter clip is zero-padded at the end, and a boolean mask records which frames are real.

The order is what makes the padding safe. Both normalisation formulas map a zero input to zero output, so an undetected block stays zero through normalisation, and padding added afterwards is also exactly zero. The classifier’s masking layer then skips any all-zero time step at no cost. Padding after normalising would have produced non-zero padded frames that the model would read as signal.

The compiled dataset is stored as three aligned arrays: the normalised sequences with shape (840, 146, 225), the integer labels with shape (840,), and the padding mask with shape (840, 146). A manifest records the class, signer, source file, and original frame count for every row, which keeps each compiled clip traceable to the recording it came from.

3.7 Phase 5: Model Training

3.7.1 Architecture

The classifier is a bidirectional LSTM built with TensorFlow and Keras:

Table 3.2: Classifier architecture

Layer	Purpose
Input (146, 225)	one normalised clip
Masking (mask value 0.0)	skips zero-padded and undetected frames
Bidirectional LSTM, 64 units	reads the motion in both directions
Dropout 0.3	regularisation [18]
Bidirectional LSTM, 32 units	summarises the sequence
Dropout 0.3	regularisation
Dense 32, ReLU	feature layer feeding the classification head
Dense 7, softmax	probability over the seven classes

The network has 192,007 trainable parameters, which is small enough to train on a laptop CPU in minutes. The layer sizes above are not asserted. Section 5.8 compares this architecture against six variants of it, under the evaluation protocol of Section 3.8.3.

3.7.2 Why This Model and Not a Larger Pretrained One

A dataset of 840 clips is small for a network trained from scratch, which raises the question of whether a pretrained model should carry more of the work. The answer is that one already does, and that the division of labour is where the data efficiency comes from.

MediaPipe Holistic is the pretrained component. It was trained on far more human pose and hand data than this project could ever collect, and it performs the part of the task that needs that scale: locating a body and two hands in an arbitrary image under arbitrary lighting. What reaches the BiLSTM is not an image but 225 already-interpreted coordinates, so the only thing left to learn is how those coordinates move over time for each phrase. That is a much smaller problem, and 192,007 parameters is a proportionate model for it.

A pretrained video classifier operating on pixels was considered and rejected for two reasons. It would need the raw video, which was deliberately never stored, so adopting one would mean re-recording the entire dataset. It would also be far too large for the deployment target, which turned out to be a phone: the whole exported classifier is 881 kilobytes, it runs without a GPU, and it is small enough to ship inside the application package alongside the landmark models.

The dataset was also enlarged during development in preference to adding model capacity, growing from four signers and 570 clips at the mid-term to eight signers and 840 clips. For a landmark-based classifier at this vocabulary size, additional signers buy more than additional parameters, because the quantity in short supply is variation between people and not model expressiveness.

Both halves of that judgement were tested afterwards rather than left as reasoning. Widening the recurrent layers to 128 and 64 units and the dense head to 64, which is 2.8 times the parameter count, moves leave-one-signer-out accuracy by four tenths of a percentage point, while the spread between held-out signers is several times that. The measurements are reported in Section 5.8.

3.7.3 Training Configuration

Training minimises sparse categorical cross-entropy with the Adam optimiser [19] at a batch size of 16, for at most 200 epochs. Class weights are computed to be inversely proportional to class frequency, which matters more for the folds than for the whole dataset, since the classes are balanced overall but a held-out signer’s contribution is not always uniform.

Two callbacks control the schedule. Early stopping monitors validation loss with a patience of 15 epochs and restores the best weights. Its minimum-improvement threshold is set to 10^{-3} and not left at zero, because with a threshold of zero any

improvement at all resets the patience counter, including a change in the fourth decimal place. Once validation loss approaches zero it keeps setting negligible new records, so training does not end on convergence; it ends whenever an optimiser instability finally breaks the streak. That made the epoch count a record of when a random fluctuation occurred instead of a property of the fit.

Learning rate reduction on plateau addresses the cause of those fluctuations, halving the learning rate after 6 epochs without improvement, down to a floor of 10^{-5} . Its patience is kept well below the early-stopping patience so that the reduced rate has a chance to take effect before training halts.

Reproducibility required more than fixing the random seed. TensorFlow’s parallel CPU reductions vary between runs, and on identical data this moved per-fold accuracy by up to 2.7 percentage points. Deterministic operations are therefore enabled explicitly, at some cost in speed, so that a reported fold accuracy is a property of the data and the recipe, not of one particular run.

3.8 Phase 6: Evaluation Methodology

3.8.1 Leave-One-Signer-Out

The headline metric is leave-one-signer-out accuracy. For each signer eligible to be held out, the model is trained from scratch on every clip from the other signers and tested only on that signer’s clips. Within each fold, 15% of the training clips are set aside as a stratified validation split for early stopping, so the held-out signer influences neither the fitted weights nor the stopping point.

A signer is eligible for holdout only if removing their clips leaves every class present in the remainder. All eight signers satisfied this, giving eight folds. Two figures are reported: the mean accuracy across folds, which estimates performance on a new person, and the pooled accuracy over all 840 predictions, where each clip is scored exactly once by a model that never saw its signer.

3.8.2 The Random-Split Contrast

A stratified 70/15/15 random split is also trained and reported. It is not an additional performance measure. Its purpose is to quantify what the signer-independent protocol is protecting against: under a random split, clips from the same person appear in both training and test, and the model can score well by recognising individuals. The gap between the two protocols is the size of that effect on this dataset, and it is discussed in Section 5.6.

3.8.3 The Architecture Comparison

The architecture in Table 3.2 fixes several numbers — two recurrent widths, a dense width, a dropout rate, and the choice of LSTM over GRU — that were settled before any accuracy had been measured. A comparison across seven variants of it is therefore also run and reported. Like the random-split contrast, it is not an additional performance measure; its purpose is to establish whether those numbers were a defensible choice rather than an arbitrary one.

The seven candidates vary one aspect of the design at a time: a smaller network (32/16/16 units), a larger one (128/64/64), a single recurrent layer instead of two, the removal of the dense head, dropout raised from 0.3 to 0.5, and the LSTM cells replaced with GRU cells. The seventh candidate is the shipped architecture itself, reconstructed to the same specification, so that it is measured on equal footing with the alternatives instead of being compared against a figure obtained some other way.

Every candidate is evaluated over the same eight leave-one-signer-out folds described above, with the same seed, the same stratified validation split, the same class weights, and the same early-stopping and learning-rate schedule. Only the architecture changes, so any difference in accuracy is attributable to it and not to a different training regime. The results are given in Section 5.8.

3.8.4 Metrics

Overall accuracy is reported alongside per-class precision, recall, and F1:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (3.1)$$

A confusion matrix pooled over all folds identifies which classes are mutually confusable. For an emergency system the two directions of error are not equivalent: failing to recognise a real phrase leaves the user without help, while announcing a phrase for a non-sign produces a false alarm. Both appear in the matrix and are discussed separately.

3.9 Phase 7: Deployment

3.9.1 Revision of the Deployment Approach

The proposal specified a browser application built with Next.js, running the model in the visitor’s browser through TensorFlow.js, with no server involved. The delivered system is an Android application that runs recognition on the phone itself,

with no server involved at use time. Two separate decisions produced that change and they are worth separating.

The first is the choice of device. A phone is what a person has with them when an emergency happens; a laptop with a browser open is not. For a tool whose value depends on being available at the moment of need, the phone is the correct target, and this was decided on those grounds. The browser route was not attempted. The change was recorded as a task deviation in the project log and accepted by the supervisor before the client was built.

The second decision concerns where inference runs, and it was revised once during development. MediaPipe Holistic, the exact landmark extractor the dataset was recorded with, exists only as a Python pipeline and has no mobile build. Running recognition on the device therefore meant reimplementing the 225-feature extraction and the dual-anchor normalisation against a different landmark source, and a numerical difference between two such implementations does not raise an error. It produces confident wrong answers, because the model receives numbers unlike anything it was trained on. On that reasoning the first working system served the model from Python over a network, so that the inference path imported the same preprocessing module the reported accuracy was measured with.

That system worked, but it made an emergency tool depend on a network connection, which the proposal had stated it would not. The on-device path was therefore built as well, and Section 3.9.2 describes what it took to make the two landmark sources agree closely enough for the trained model to survive the substitution. The delivered application runs on the device by default and needs no network at all. The server is retained as a selectable fallback for a phone too slow to extract landmarks in real time, and as the verification path described in Appendix F.

The cost did not disappear; it changed shape. The on-device path is a reimplementation rather than the same code, and the guarantee that inference and training preprocess identically is now enforced by tests rather than by construction. That is a weaker guarantee, and it is recorded among the limitations in Chapter 6.

3.9.2 On-Device Recognition

The phone extracts landmarks with the MediaPipe Pose Landmarker and Hand Landmarker, the mobile-supported successors to Holistic, and assembles them into the same 225-value vector in the same order. Because Holistic assigns hands implicitly and the two mobile landmarkers do not, the on-device assembler reproduces that assignment by matching each detected hand to the nearer pose wrist, and

carries a flag recording whether MediaPipe’s own handedness classifier agreed. A disagreement would mean the two hand blocks had swapped, which is the one error in this pipeline that produces plausible output rather than an obvious failure.

Normalisation and length standardisation are ported to the client and pinned to the Python implementation by generated fixtures: a script exports the intermediate arrays for a set of real clips, and the client’s test suite fails if its own output diverges. The port also reproduces one detail that would otherwise drift silently, namely that the subsampling index uses round-half-to-even, as NumPy does, rather than the round-half-up of the client language.

Two distortions were found once the path could be measured against recorded clips, and both are corrected before any of the shared preprocessing runs.

The first is geometric. MediaPipe reports each coordinate as a fraction of the frame’s width and height, so the aspect ratio of the frame decides how much vertical distance one unit represents. The dataset was recorded through an unconfigured webcam in landscape, and the phone captures in portrait. Without correction the on-device skeleton arrives at the correct width and roughly half the correct height, which the dual-anchor normalisation cannot repair because it divides by a single, largely horizontal, scalar. Rescaling the vertical coordinates by the ratio of the two aspect ratios removes it.

The second is temporal. Pose and hand landmarking on a phone processor does not sustain the capture rate of the training data, so a six-second phrase can arrive as forty-five frames where the recordings held about ninety-five. Since the model reads frame count as duration, that is not a quality loss but a different sign: measured before the correction, genuine `need_ambulance` clips were classified as `none` with full confidence. The client therefore resamples the completed clip along time to the frame count the training capture rate would have produced over the measured wall-clock duration of the recording. Interpolation cannot recover motion the low frame rate never sampled; it restores the temporal scale, which is what the model is sensitive to. Interpolation never crosses a missing block, since averaging a detected hand against an absent one would invent a landmark halfway to the origin.

3.9.3 Model Export

The trained model is converted to TensorFlow Lite [20], which is what allows it to run on the phone at all and, on the fallback server, to run without a full TensorFlow installation, reducing that deployed image from about three gigabytes to one. The conversion required a workaround: on the pinned TensorFlow and Keras versions used here, the standard converter entry points abort the host process inside the

compiler backend instead of raising a catchable exception. Freezing the model’s variables to constants before conversion avoids the failure.

Because a silent conversion error would be indistinguishable from a model that simply performs worse, the export is verified and not assumed. The converted model and the original are run on 30 real clips and compared; the export is rejected if they disagree. The measured agreement is reported in Section 5.10.

3.9.4 The Fallback Inference Server

The server predates the on-device path and is retained for two purposes: it serves phones that cannot extract landmarks in real time, and it is the path by which the deployed decision can be checked against the offline pipeline without a phone at all.

It is a FastAPI application exposing a WebSocket endpoint. A client opens a session, streams JPEG frames while the user signs, and sends a completion message; the server replies with the decision. Landmark extraction costs about 38 milliseconds per frame and accounts for almost the entire pipeline cost, so streaming frames during signing overlaps that work with the signing itself. When the client signals completion, only normalisation, standardisation, and the forward pass remain, which together take about 15 milliseconds. Uploading a finished recording instead would have serialised the two and added several seconds of silence before every result.

Each session holds its own MediaPipe graph, which occupies about 150 megabytes, so concurrency is bounded by memory, not processor time. A semaphore caps simultaneous sessions and refuses further connections with an explicit error instead of allowing the process to be killed for exhausting memory. Requests are authenticated with a shared key compared using a constant-time function, so that a wrong key cannot be narrowed down by timing the response. The health endpoint is left unauthenticated so that container health checks and the client’s reachability test continue to work.

3.9.5 Frame Rate as a Correctness Constraint

Because the model reads frame count as duration, a capture rate that differs from the training data’s presents the model with a temporal scale it has never seen. The two deployment paths meet this constraint from opposite directions, because they fail in opposite directions.

A network client can capture faster than the training data: at 30 frames per second it produces twice as many frames for the same sign. The server therefore discards incoming frames down to the 15.7 frames per second of the training data

before running detection, using an accumulator that tracks when the next frame is due. A simpler test of whether enough time has elapsed since the last accepted frame aliases a 16 frames-per-second client down to 8. The client caps its own send rate at 16 frames per second, slightly above the target so that jitter does not starve the server below it.

On the device the problem is the reverse: landmark extraction is the bottleneck and the phone captures more slowly than the training data, so there is nothing to discard. The correction there is the temporal resampling of Section 3.9.2, applied to the finished clip against its measured duration. Both paths therefore deliver a clip whose frame count means what it meant during recording, which is the property the model actually depends on.

3.9.6 Mobile Client

The Android client captures camera frames, converts them, runs the recogniser, and speaks the result. In the fallback server mode it encodes and transmits frames instead of recognising them.

Frame conversion is the client’s bottleneck, since it is pure Dart. Colour conversion, rotation, mirroring, and downscaling are folded into a single pass over the output pixels, avoiding the intermediate full-resolution buffers a naive implementation allocates. The work runs on a persistent background isolate. When a frame is already in flight the next one is dropped instead of queued, since a queue would only add latency to a result the user is waiting for. On the on-device path each frame becomes a 480-pixel-wide raw image handed to the landmarks; on the server path it is downscaled to 320 pixels and encoded as JPEG with chroma subsampling, on the grounds that landmarks are derived from luminance structure and not chroma resolution.

Two details are recorded because they are easy to get wrong. The application is locked to portrait orientation and rotates each frame by the camera’s sensor orientation, because Android delivers frames in sensor orientation and MediaPipe detects nothing at all in an image of a person lying sideways. Every frame is mirrored to match the horizontal flip the recorder applied, and the mirror is applied for both front and rear cameras: the lens faces the signer either way, so the signer’s right hand appears on the image-left in both cases, and making the mirror depend on the camera would swap the two hand blocks in the feature vector for one of them. On the server path the frames go unmirrored with a flag instructing the server to apply the same flip, which keeps the single implementation of it on the side that owns the feature vector.

The client also writes the raw landmark clip for each attempt to device stor-

age. The recorder saved landmarks so that a bad recording could be inspected afterwards; the phone originally did not, and that is precisely what made the two distortions of Section 3.9.2 difficult to locate. Both were diagnosed by pulling a clip off the device and running it through the Python pipeline beside a recorded one.

3.9.7 The Decision Rule

Three outcomes are possible:

1. If the predicted class is one of the six emergency phrases and the softmax confidence reaches 0.75, the result is **accepted**. The phrase is displayed and spoken.
2. If the predicted class is **none**, the result is **unknown**. The classifier has positively identified the input as not being one of the six phrases, which is a different statement from being unsure, and the user is told so.
3. Otherwise the result is **low_confidence**: the top class is an emergency phrase but the model does not carry enough probability on it, and the user is told the input was not recognised clearly.

Only **accepted** speaks. The two refusals are distinguished because they call for different responses from the user — **unknown** means the system saw motion it does not treat as a target phrase, while **low_confidence** means it saw something phrase-like and would not commit — but neither produces audio, so a wrong announcement and no announcement stay on the same footing.

Separating **unknown** from **low_confidence** also removes a failure mode that the two-outcome design would have carried into deployment. The **none** class has a human-readable label like any other class, so a confidently recognised non-sign would otherwise satisfy the acceptance test and be announced aloud, which is the opposite of what the negative class was added to achieve. Routing **none** to its own refusing status is what prevents that.

The best guess and the top three probabilities are returned in every case, including both refusals, which is what makes a failure diagnosable during a demonstration.

CHAPTER 4

SYSTEM DESIGN

This chapter records the requirements the system was built against, the feasibility assessment made before construction, and the design of the delivered system. The diagrams describe what was built and not what was proposed; the block diagram in particular differs from the one in the proposal, for the reasons given in Section 3.9.1.

4.1 Requirement Specification

4.1.1 Software Requirements

Table 4.1: Software used, and the role of each component

Component	Role in the system
Python 3.11	Recording, preprocessing, training, and the inference server. The version is pinned because MediaPipe and TensorFlow resolve to a compatible set only on 3.11.
MediaPipe Holistic	Pretrained pose and hand tracking. Produces the 225 landmark values per frame, offline during recording and again on the fallback server at inference.
MediaPipe Pose and Hand Landmarker	The mobile-supported successors to Holistic. Produce the same 225 values on the phone, since Holistic has no mobile build.
OpenCV	Camera capture, frame decoding, colour conversion, and the horizontal mirror applied before detection.
TensorFlow / Keras	Building and training the BiLSTM, and exporting it.
NumPy	Landmark arrays, normalisation arithmetic, and the compiled dataset tensors.
scikit-learn	Stratified splits, class weights, and the confusion matrix and classification report.
Matplotlib	Training curves, the confusion matrix, and the frame-count histogram.
TensorFlow Lite	The deployed model format. Runs on the phone, and on the fallback server through a runtime that does not require full TensorFlow.
FastAPI and Uvicorn	The fallback inference service and its WebSocket endpoint.
Docker	Packaging the server with its model and runtime so deployment does not depend on the host environment.
Flutter and Dart	The Android client: camera capture, frame conversion, the ported preprocessing, the WebSocket protocol for the fallback path, and speech output.
Kotlin	The Android bridge that runs the two landmark models and assembles the 225-vector on the device.
Git and GitHub	Version control for the code, and the mechanism by which eight signers' recordings were merged into one dataset.

4.1.2 Hardware Requirements

Training and evaluation ran on an ordinary laptop with no GPU. The model has 192,007 parameters and each evaluation fold completes in minutes on a processor alone, so no accelerator was required at any stage. Data collection needed only a webcam. At use time the system needs an Android phone with a camera and

roughly 18 megabytes of storage for the bundled models, and nothing else: no network connection, no server, and no accelerator, since the landmarks fall back to the processor when no GPU delegate is available. The fallback server path additionally needs a machine to run the container and a network the phone can reach.

4.1.3 Functional Requirements

1. **Dataset recording.** The system shall record clips of each target phrase, extract 225 landmark values per frame, and store each clip with its class, signer identity, frame count, and per-block detection rates.
2. **Preprocessing.** The system shall normalise every clip against the shoulder-midpoint and wrist anchors and standardise it to the fixed sequence length, by a definition held in one place, with any reimplementations of it verified against that definition rather than trusted.
3. **Phrase recognition.** Given a captured sequence, the system shall produce a probability distribution over the seven classes.
4. **On-device operation.** The system shall perform landmark extraction and classification on the user's phone, so that recognition does not depend on a network connection at the moment of use.
5. **Rejection of unknown input.** The system shall be trained with an explicit negative class covering non-signing motion, so that an input unlike any of the six target phrases has a class to be assigned to instead of being forced into one of them, and shall report such an input as unrecognised rather than announcing it.
6. **Confidence thresholding.** The system shall emit a phrase only when confidence reaches the configured threshold, and shall remain silent otherwise.
7. **Text and speech output.** On an accepted recognition the system shall display the phrase on screen and speak it aloud.
8. **Live capture.** The client shall process frames while the user signs and present the result when signing ends, without the user needing to time or trim the recording.
9. **Evaluation reporting.** The system shall report signer-independent accuracy, a pooled confusion matrix, and per-class precision, recall, and F1.

4.1.4 Non-Functional Requirements

1. **Response time.** A result shall be returned promptly enough that the interaction feels immediate, which requires overlapping landmark extraction with signing instead of performing it after capture.
2. **Silence over error.** A wrong announcement is more harmful than no announcement. The system shall prefer refusing to answer.
3. **Train and serve equivalence.** The features presented to the model at inference shall be produced by the same procedure that produced the training features. Where the deployment platform makes sharing the code impossible, the reimplementation shall be pinned to the original by generated fixtures that fail on divergence.
4. **Cost.** The system shall use only free and open-source components and shall require no GPU or paid service.
5. **Privacy.** No video of any signer shall be stored, and no image shall leave the user's phone during recognition. The dataset holds landmark coordinates only.
6. **Usability under stress.** The interface shall need one action to start and one to finish, with large readable output, and shall require no configuration before a first recognition.
7. **Diagnosability.** The client shall display what the recogniser observed per frame, so that a failure during use can be attributed instead of guessed at, and shall retain the landmark clip behind each attempt for later inspection.
8. **Maintainability.** The feature layout, normalisation anchors, sequence length, and thresholds shall be defined in one place and imported everywhere else.

4.2 Feasibility Assessment

4.2.1 Economic Feasibility

Every component is free and open-source. Development used laptops the team already owned, and no GPU, paid dataset, or cloud service was purchased at any point. Because recognition runs on the phone, the delivered system has no recurring cost at all: there is no server to keep running and no traffic to pay for. The optional fallback server, if deployed, is the only component that would carry one.

4.2.2 Technical Feasibility

The enabling technologies are mature. MediaPipe provides pretrained tracking that needs no training data of its own, on the desktop and on the phone alike, and the BiLSTM is a well-understood architecture for sequence classification. Reducing each frame to 225 coordinates lowers the dimensionality enough that a few hundred clips suffice, which is what made a custom dataset viable within the project timeline, and it leaves a classifier small enough to run on a phone. The principal risk was never the model; it was whether an authentic dataset could be collected at all, which depended on access to the Deaf community.

4.2.3 Operational Feasibility

The interaction is two taps: one to begin signing and one to end. The system needs no specialised hardware on the user’s side beyond a phone, no network at the moment of use, no account, and no configuration before the first recognition. It addresses a need the Deaf organisations consulted during this project confirmed is real, and can be demonstrated live. The main operational weakness is now the speed of landmark extraction on modest hardware, which is what the temporal correction of Section 3.9.2 exists to absorb and what the fallback server exists to escape.

4.3 Use Case Diagram

Figure 4.1 shows the actors and their interactions. The Deaf Signer starts a session and performs an emergency phrase. The Hearing Person is the recipient of the output, reading the displayed phrase and hearing it spoken; they do not operate the system. The Developer role covers the offline work: recording and labelling the dataset, training and evaluating the model, and packaging the trained model into the application.

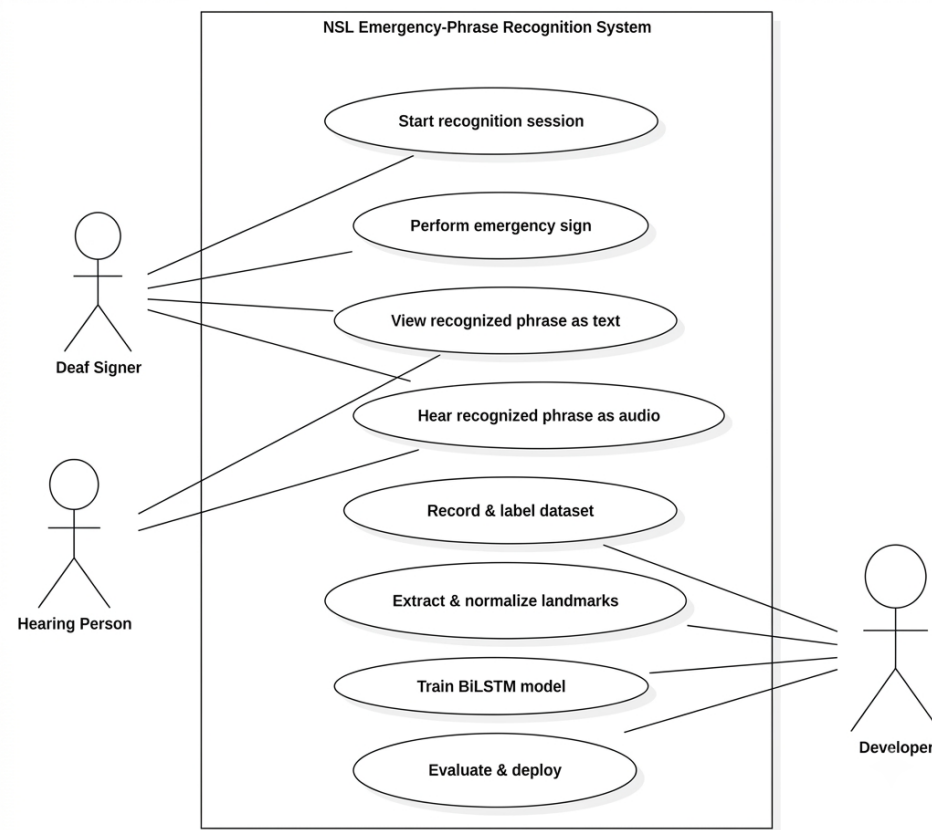


Figure 4.1: Use case diagram of the NSL emergency-phrase recognition system

4.4 System Block Diagram

The system has three parts. An offline path builds the model, the phone reproduces the offline preprocessing at inference time and runs the classifier, and a server offers the same path over a network for devices that cannot. Figure 4.2 shows all three and the preprocessing definition that connects them.

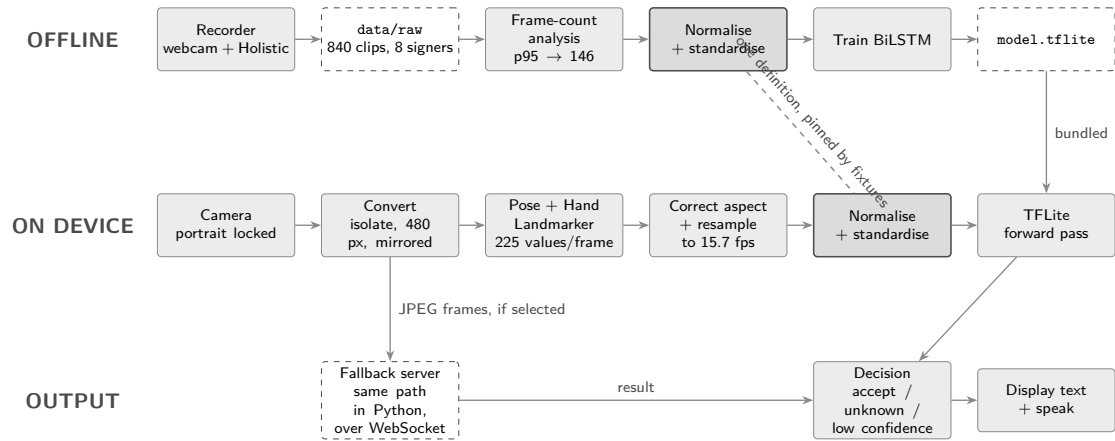


Figure 4.2: System block diagram. The offline path builds the model; the phone reproduces the same preprocessing and runs the classifier; the server offers the same path over a network as a fallback. The two shaded boxes are the same preprocessing definition.

The preprocessing definition is the load-bearing element of the design. The training scripts and the fallback server import one module; the phone carries a port of it, and that port is pinned to the original by fixtures generated from the same module, so a divergence fails a test rather than reaching a user as a confident wrong answer.

4.5 Data Flow Diagrams

4.5.1 Level 0

Figure 4.3 shows the system as one process with its external entities. The signer supplies gestures and receives text and speech. The hearing recipient receives the same output. The developer supplies labelled recordings and the trained artefacts.

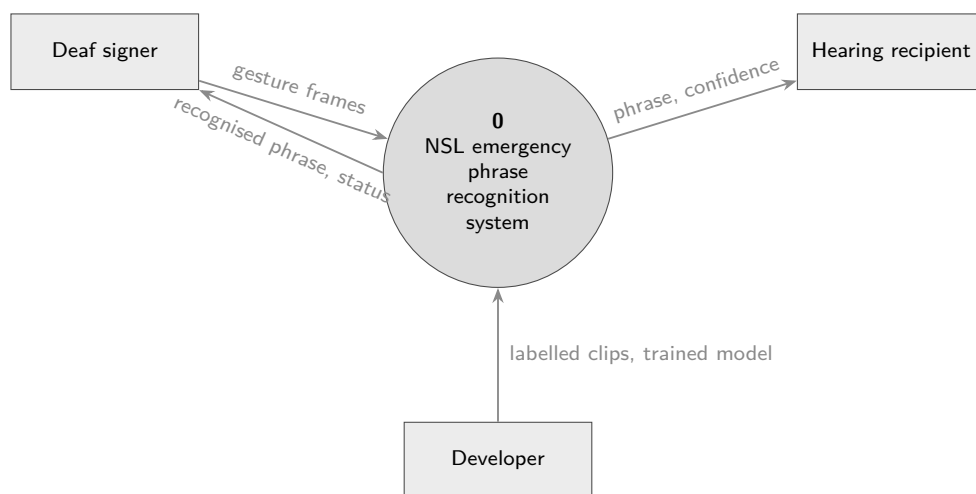


Figure 4.3: Level 0 data flow diagram

4.5.2 Level 1

Figure 4.4 decomposes the system into the processes that carry out recognition, and shows the two data stores.

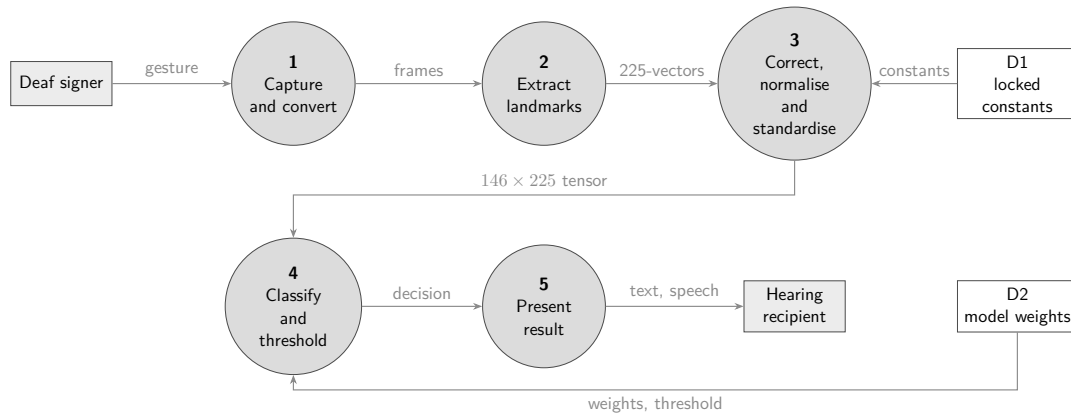


Figure 4.4: Level 1 data flow diagram

4.6 Sequence Diagram

Figure 4.5 shows one recognition on the device, from the tap that starts it to the spoken result. The loop is the important part: frames are landmarked while the user is still signing, so the cost of extraction is paid during the sign and not after it. Only the two corrections, the normalisation, and the forward pass remain once the user stops.

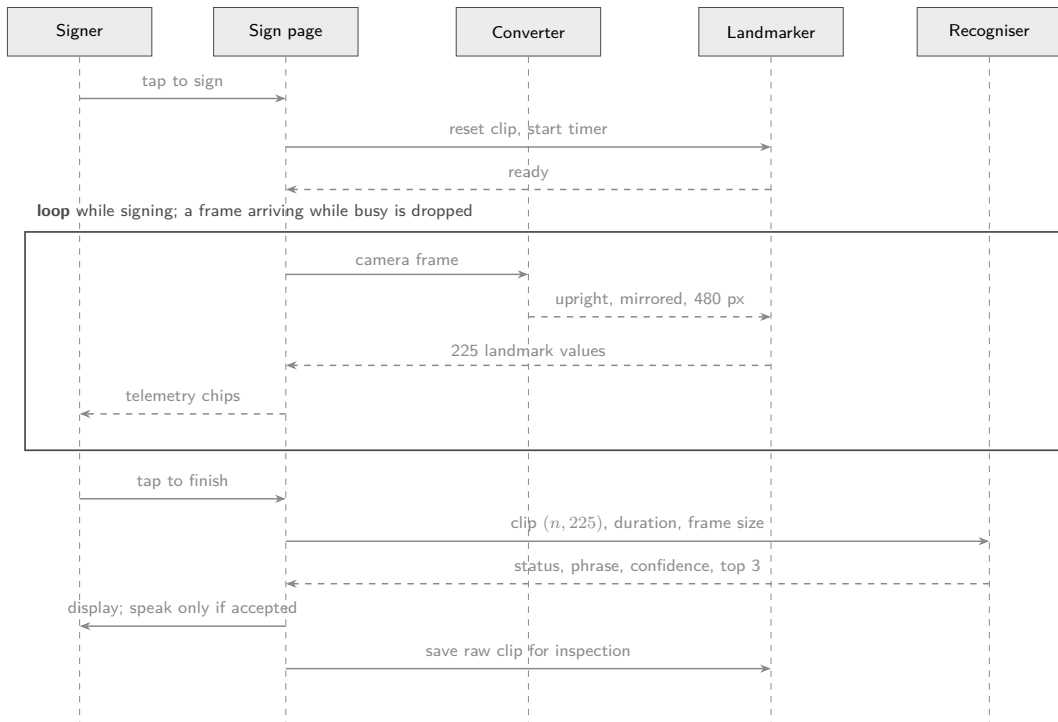


Figure 4.5: Sequence diagram for one on-device recognition

4.7 Class Diagram

Figure 4.6 shows the principal modules and classes. The Python package on the left is imported by both the training scripts and the fallback server. The client package on the right carries a port of the same preprocessing contract, drawn with a heavier outline, and the fixture generator that pins the port to the original is shown between them.

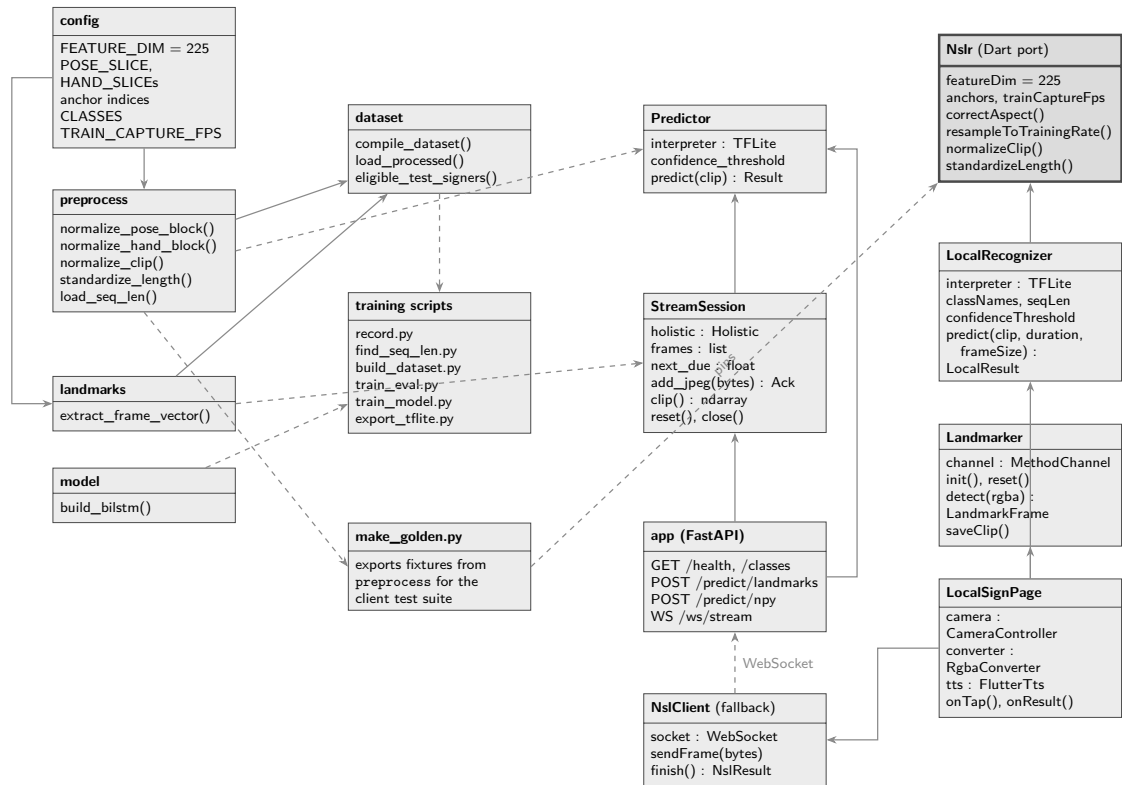


Figure 4.6: Class and module diagram. Solid arrows are use relationships; dashed arrows are imports or verification relationships. The heavier box on the right is the client's port of the Python preprocessing contract, pinned to it by generated fixtures.

4.8 Activity Diagram

Figure 4.7 shows the control flow of one recognition attempt, including the decision point that can end it without output.

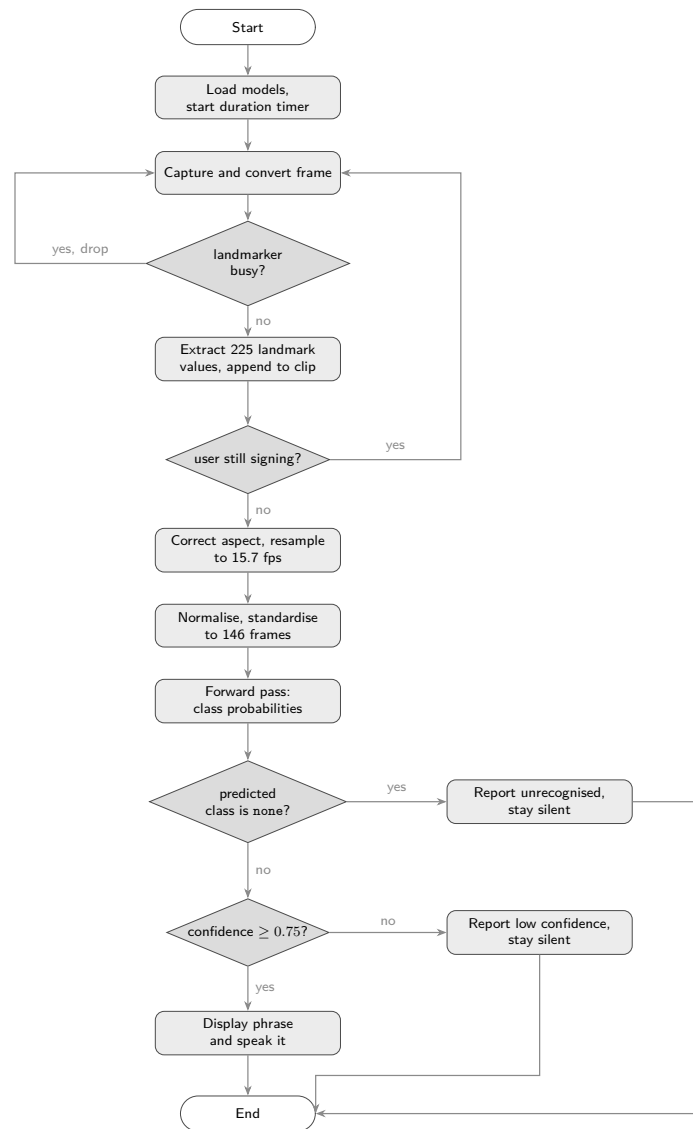


Figure 4.7: Activity diagram for one on-device recognition attempt

CHAPTER 5

RESULT AND DISCUSSION

BEFORE SUBMISSION – DELETE THIS BOX

Every number and figure in this chapter comes from `results/metrics.json`, `results/model_meta.json` and `data/processed/`, at `seq_len = 146` over 840 clips, and the four generated figures have been refreshed from that same run. One defect in the source data is worth knowing about.

`results/metrics.json` carries a malformed `confusion_matrix`: the `call_police` row sums to 121 against a support of 120, and the `cant_breathe` row has nine columns instead of seven. Neither can come from the code that writes the file, so it has been hand edited at some point. Table 5.5 was therefore reconstructed from the per-class precision and recall figures, which are self-consistent and pin every column sum, and it agrees cell for cell with Figure 5.4, which was rendered directly from the run. Do not patch the JSON — re-run the pipeline and take the matrix from the fresh output.

On the final retrain, run in this order — `find_seq_len.py`, `build_dataset.py`, `train_eval.py`, `train_model.py`, `export_tflite.py` — then copy the three model assets into `app/assets/models/` and re-run the client test suite, or the phone will carry a model that does not match this chapter. Then copy the four regenerated figures into `img/` and update every number here, in the abstract, and in Chapter 6.

This chapter reports what the system achieves and where it fails. The dataset is described first, then the recognition results under the signer-independent protocol, then the deployment measurements, and a discussion of the remaining weaknesses.

5.1 Experiment Setup

All training and evaluation ran on an ordinary laptop with no GPU, under Python 3.11 with TensorFlow, MediaPipe, NumPy, and scikit-learn pinned to a mutually compatible set. The classifier has 192,007 parameters, and a single evaluation fold completes in minutes on the processor alone.

The random seed was fixed at 42 for every run. Fixing the seed by itself proved insufficient: TensorFlow’s parallel processor reductions vary between runs, and on identical data this moved per-fold accuracy by as much as 2.7 percentage points. Deterministic operations were therefore enabled explicitly, so the accuracies below are reproducible from the same data and code, not one sample from a distribution of runs.

5.2 Dataset Composition

The final dataset holds 840 clips, distributed exactly evenly at 120 clips in each of the seven classes. Table 5.1 gives the breakdown by class and signer.

Table 5.1: Clips by class and signer in the final dataset

Class	s01	s02	s03	s04	s05	s06	s07	s08	Total
building_on_fire	20	20	20	20	10	10	10	10	120
call_police	20	20	20	20	10	10	10	10	120
cant_breathe	20	20	20	20	10	10	10	10	120
help_danger	20	20	20	20	10	10	10	10	120
need_ambulance	20	20	20	20	10	10	10	10	120
need_toilet	20	20	20	20	10	10	10	10	120
none	20	20	20	20	0	20	10	10	120
Total	140	140	140	140	60	80	70	70	840

The four signers who recorded 20 clips per class were team members, who could record repeatedly. The four who recorded 10 per class include the Deaf consultants and additional volunteers, whose available time was shorter. Class balance across the seven classes was maintained deliberately, since an imbalanced set would have let the classifier gain accuracy by favouring whichever class was most common.

The **none** row is the one departure from the otherwise regular pattern. Signer 05 recorded no negatives and signer 06 recorded twice the usual number, so the class total still comes to 120 while those two signers contribute 60 and 80 clips overall rather than 70 each. This matters for one thing only, and it is worth stating before the fold results: signer 05’s held-out fold is scored on six classes, not seven, because that signer has no **none** clips to be tested on. Since **none** is the hardest class, that fold is measured on a slightly easier problem than the other seven.

Eight signers is a small number, and it is the principal limitation on how much the reported accuracy can be trusted to generalise. This is discussed in Section 5.14.

5.3 Sequence Length and Standardisation

Frame counts across the 840 clips range from 30 to 217, with a median of 89 and a mean of 91.7. Figure 5.1 shows the distribution. The 95th percentile falls at 146 frames, which became the fixed sequence length.

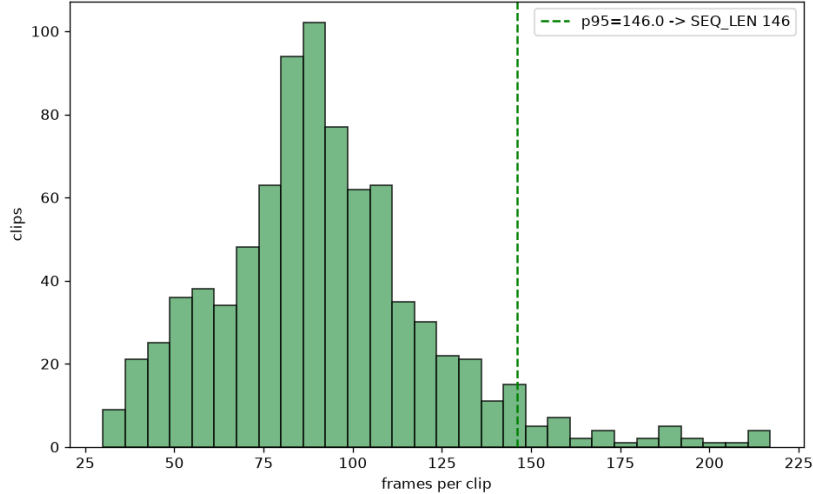


Figure 5.1: Distribution of raw clip frame counts across the 840 recorded clips, with the 95th-percentile cutoff that fixes the sequence length at 146 frames

At a capture rate of 15.7 frames per second, 146 frames corresponds to about 9.3 seconds of signing, and the median clip of 89 frames to about 5.7 seconds. Table 5.2 shows how the standardisation step treated the clips.

Table 5.2: How the 840 clips were standardised to 146 frames

Mode	Clips	Treatment
Padded (< 146 frames)	794	zero-padded at the end, with a mask recording the real frames
Subsampled (> 146 frames)	39	sampled at 146 evenly spaced positions
Exact (146 frames)	7	used unchanged
Total	840	

Of the 122,640 frame slots in the compiled tensor, 76,018 hold real recorded frames and the remaining 38.0% are zero-padding that the masking layer skips. Only 39 clips were long enough to be compressed, which matters for a reason returned to in Section 5.14: below 146 frames the model receives duration information directly in the frame count, and above it that information is normalised away.

5.4 Training Behaviour

Every fold converged and stopped on the early-stopping criterion before exhausting the 200-epoch budget. Table 5.3 reports where each fold stopped alongside its

accuracy. The spread from 29 to 45 epochs reflects how quickly validation loss stopped improving on each training subset.

Figure 5.2 shows accuracy per epoch for all eight folds and Figure 5.3 the corresponding losses. Each accuracy panel carries a dashed line at that fold's held-out score. The gap between the training and validation curves, both drawn from signers inside the training set, and that dashed line is the cost of meeting a signer the model has never seen. Presenting all eight folds shows whether the folds behaved consistently, which a single representative curve cannot.

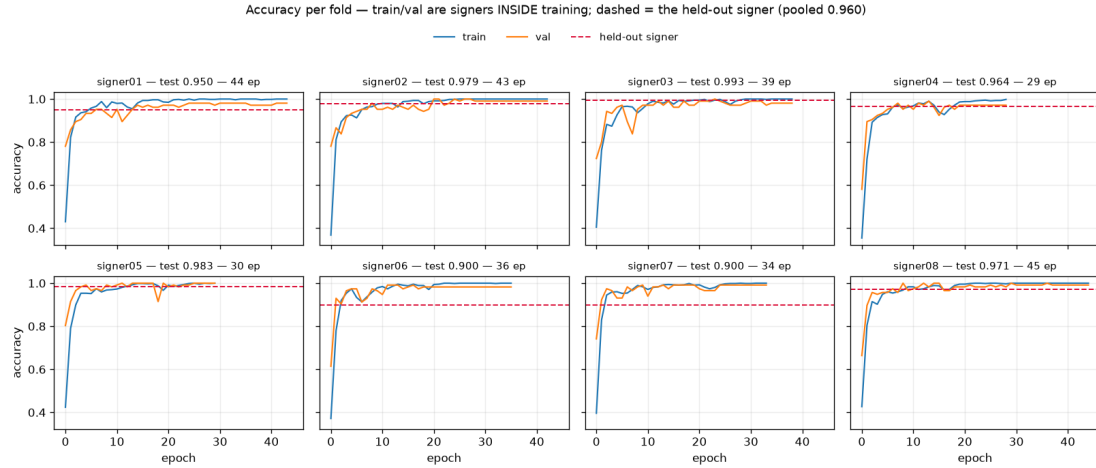


Figure 5.2: Accuracy per epoch for each leave-one-signer-out fold. Training and validation curves are drawn from signers inside the training set; the dashed line marks the held-out signer's score.

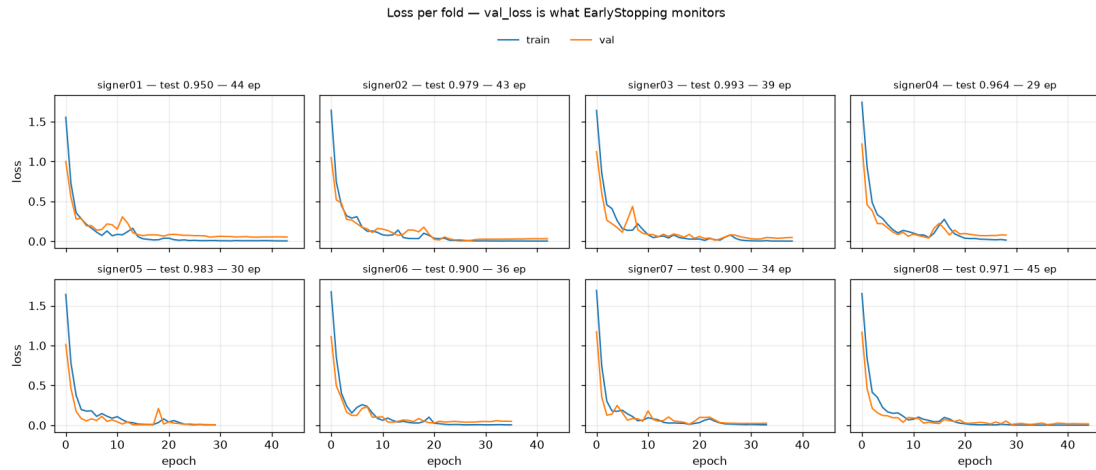


Figure 5.3: Loss per epoch for each fold. Validation loss is the quantity early stopping monitors.

5.5 Signer-Independent Recognition Results

All eight signers were eligible for holdout, since removing any one of them left every class present in the remainder. Table 5.3 gives the result of each fold.

Table 5.3: Leave-one-signer-out results. Each fold trains on seven signers and tests on the eighth.

Held-out signer	Test clips	Accuracy	Epochs to stop
signer01	140	0.9500	44
signer02	140	0.9786	43
signer03	140	0.9929	39
signer04	140	0.9643	29
signer05	60	0.9833	30
signer06	80	0.9000	36
signer07	70	0.9000	34
signer08	70	0.9714	45
Mean across folds		0.9551 ± 0.0340	
Pooled over 840 clips		0.9595	

Mean accuracy across folds is 95.51% with a standard deviation of 3.40 percentage points. Pooling all 840 predictions, each made by a model that had not seen its signer, gives 95.95%.

The spread between folds is more informative than the mean. The best fold reaches 99.29% and the two worst sit at 90.00%, a range of 9.3 points. Since each fold differs only in which signer was withheld, that spread is a direct measurement of how much recognition quality depends on the individual. Reporting the mean alone would hide this; a system deployed to the public will meet signers at both ends of that range.

Two of the folds need a caveat rather than a straight reading. Signer 05 scores 98.33%, but on 60 clips that carry no **none** examples, so that fold never faces the hardest class; it is the least comparable number in the table. Signer 06 scores 90.00% on 80 clips, of which 20 are negatives rather than the usual 10, so that fold faces the hardest class twice as often as most. The two effects push in opposite directions and both trace back to the same irregularity in Table 5.1. Signer 07, at the same 90.00% on a conventional 70 clips, is the more honest worst case.

5.5.1 Per-Class Performance

Table 5.4 gives precision, recall, and F1 for each class, pooled over all folds.

Table 5.4: Per-class performance, pooled over all eight folds

Class	Precision	Recall	F1	Support
building_on_fire	0.9500	0.9500	0.9500	120
call_police	0.9836	1.0000	0.9917	120
cant_breathe	0.9520	0.9917	0.9714	120
help_danger	0.9672	0.9833	0.9752	120
need_ambulance	0.9524	1.0000	0.9756	120
need_toilet	0.9833	0.9833	0.9833	120
none	0.9238	0.8083	0.8622	120
Macro average	0.9589	0.9595	0.9585	840

The six emergency phrases achieve F1 between 0.950 and 0.992. Two of them, `call_police` and `need_ambulance`, are never missed at all: every one of their 240 clips was recognised, and what costs them precision is other classes being read as them rather than any failure of their own. The negative class is clearly the weakest at 0.862, with a recall of 0.808.

That asymmetry is expected. The six phrases are each a specific, repeatable performance, and the model has 120 examples of each. The `none` class is not a phrase; it is everything else, sampled 120 times from an unbounded space of possible non-signs. No number of examples makes that category as learnable as the six, and its recall would be expected to remain the lowest even with far more data.

5.5.2 Confusion Matrix

Figure 5.4 shows the pooled confusion matrix and Table 5.5 the same counts numerically.

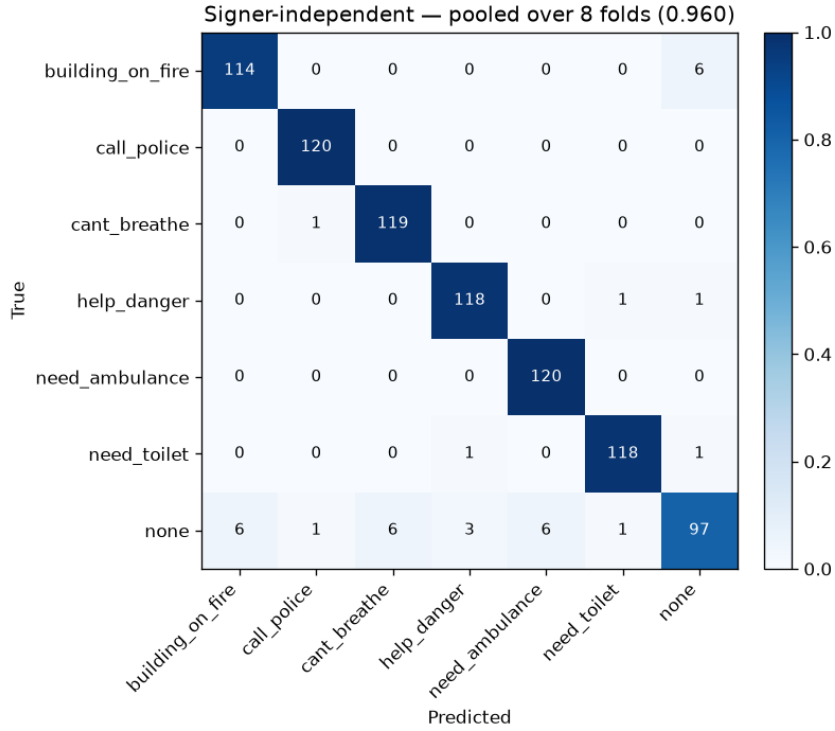


Figure 5.4: Confusion matrix pooled over all eight leave-one-signer-out folds

Table 5.5: Pooled confusion matrix. Rows are the true class, columns the prediction.

	<i>fire</i>	<i>police</i>	<i>breathe</i>	<i>help</i>	<i>ambulance</i>	<i>toilet</i>	<i>none</i>
building_on_fire	114	0	0	0	0	0	6
call_police	0	120	0	0	0	0	0
cant_breathe	0	1	119	0	0	0	0
help_danger	0	0	0	118	0	1	1
need_ambulance	0	0	0	0	120	0	0
need_toilet	0	0	0	1	0	118	1
none	6	1	6	3	6	1	97

Thirty-four of the 840 predictions are wrong, and they fall into three groups that carry very different consequences.

Twenty-three errors are non-signs predicted as emergency phrases, in the bottom row. These are the most serious errors the system can make, because each one is an occasion where the system would announce an emergency that nobody signed, and they account for two thirds of all the errors in the matrix. They are spread across all six phrases, which suggests the cause is a genuine resemblance

between some non-signing motion and the opening of a phrase rather than one specific confusable pair. This single row is where the system’s remaining weakness lives, and Section 5.14 returns to it.

Eight errors are emergency phrases predicted as `none`, in the last column: six for `building_on_fire`, one for `help_danger`, and one for `need_toilet`. Here the system stays silent when the user did sign. This leaves the user without help, though it does not produce a false alarm, and in a real deployment the user would notice the absence of a response and sign again.

Three errors are one emergency phrase confused with another, and each is isolated: one `cant_breathe` read as `call_police`, one `help_danger` as `need_toilet`, and one `need_toilet` as `help_danger`. A wrong phrase is worse than silence, because it sends the wrong help, so it is worth recording that this category is now the smallest of the three and contains no repeated pair. Confusion between the six phrases themselves is close to solved on this dataset; distinguishing them from everything that is not a sign is not.

Notably, no phrase is confused with any other in more than one clip, and `call_police` and `need_ambulance` are never missed at all. Every error in their columns is a non-sign or another phrase arriving from elsewhere.

5.6 What the Random Split Shows

Under a stratified 70/15/15 random split, the same architecture and the same training recipe reach 98.41% accuracy on 126 held-out clips. This is 2.5 percentage points above the pooled signer-independent figure, and 2.9 points above the mean across folds.

That gap is the point of reporting the number. Under a random split, clips from the same person appear in both training and test, so the model can score well by recognising the individual as much as the sign. The signer-independent protocol removes that possibility, and the accuracy falls. The 98.41% figure describes how well the system recognises phrases performed by people it has already been trained on, which is not the situation any real user is in.

Table 5.6: The two evaluation protocols compared

Protocol	Accuracy	What it measures
Random split, stratified 70/15/15	0.9841	performance on known signers
Signer-independent, mean of 8 folds	0.9551	performance on a new signer
Signer-independent, pooled	0.9595	performance on a new signer

This is worth stating because published NSL recognition results are generally

reported without separating signers, including the closest prior landmark-based work [6]. A comparison of headline accuracies across papers is therefore not meaningful unless the protocols match. On this dataset the difference between the two protocols is 2.5 points, and a project reporting only the higher number would not be doing anything unusual by the standards of the field.

5.7 Generalisation and Overfitting

A model with 192,007 parameters trained on 840 clips has enough capacity to memorise its training set, so whether it has done so is a question the evaluation has to answer rather than assume. Three accuracies are available for each fold, and the distance between them is what the question turns on.

Table 5.7: The three levels of accuracy, and what the gap between each pair means

Measured on	Drawn from	A large gap to the next level means
Training clips	signers inside training	the model is fitting its own examples
Validation clips	signers inside training, clips held out	memorisation of individual clips
Held-out signer	a person absent from training	memorisation of the people, not the signs

Four pieces of evidence bear on overfitting.

First, no fold ran to the epoch limit. Early stopping halted every fold between 29 and 45 epochs against a budget of 200, monitoring validation loss with the best weights restored. A model still improving on held-out data at the point it stops has not begun to memorise; a model that needed the full 200 epochs and kept its final weights would be the worrying case.

Second, the regularisation is not nominal. Two dropout layers at 0.3 sit between the recurrent layers, the learning rate halves after six epochs without improvement, and the masking layer removes padded frames from the computation so they cannot be fitted as signal.

Third, the per-fold spread is contained. Eight independently trained models, each missing a different signer, produce accuracies between 0.9000 and 0.9929 with a standard deviation of 3.40 percentage points. A model that had memorised its training signers would be expected to collapse on some folds and not others; the worst fold here is nine points below the best, not thirty. The spread is nonetheless wider than the differences between any two architectures tested in Section 5.8, which is the finding this chapter keeps returning to.

Fourth, and least comfortably, the random-split comparison puts a number on the memorisation that does remain. The 2.5-point gap between 98.41% and 95.95% is the portion of a random-split score that comes from recognising people rather than phrases. It is small, but it is not zero, and it is the reason the signer-independent figure is the one quoted throughout this report.

Taken together these say the model generalises to a new signer with a loss of roughly two and a half points against a same-signer measurement, and that it is not overfitting in the sense of having memorised its training clips. What they do not say is that it generalises to a new signer drawn from outside the population of eight, which is a separate claim and is addressed in Section 5.14.

5.8 Architecture Comparison

The layer sizes in Table 3.2 were settled before any of the accuracies above existed, and nothing reported so far establishes that they were the right choice. The protocol in Section 3.8.3 tests them: seven variants of the architecture, each evaluated over the same eight leave-one-signer-out folds with the same seed, split, and callbacks, so that a difference between them is a property of the architecture and not of the training regime. Table 5.8 gives the result and Figure 5.5 shows the same numbers as intervals.

Table 5.8: Architecture variants under the leave-one-signer-out protocol, ranked by mean accuracy. Units are given as first recurrent layer / second recurrent layer / dense head; a dash marks a layer the variant does not have.

Architecture	Units	Cell	Dropout	Params	Mean	95% CI	Epochs
gru	64/32/32	GRU	0.3	145,159	0.972	[0.962, 0.982]	36
bigger	128/64/64	LSTM	0.3	535,559	0.965	[0.947, 0.983]	43
current	64/32/32	LSTM	0.3	192,007	0.961	[0.944, 0.978]	42
single_layer	64/-/32	LSTM	0.3	152,839	0.960	[0.939, 0.980]	43
no_dense_head	64/32/-	LSTM	0.3	190,151	0.952	[0.930, 0.974]	40
high_dropout	64/32/32	LSTM	0.5	192,007	0.947	[0.925, 0.970]	50
smaller	32/16/16	LSTM	0.3	77,063	0.944	[0.911, 0.976]	51

BEFORE SUBMISSION – DELETE THIS BOX

The **current** row of Table 5.8 is the shipped architecture measured through this sweep, and it reports mean 0.961 with a standard deviation of 0.024, against 0.9551 and 0.0340 from `results/metrics.json` in Table 5.3. The two use the same protocol, seed, splits, callbacks and sequence length, so they ought to agree, and they differ by 0.6 points.

The difference is the runner, not the model. The sweep is executed by `scripts/sweep_parallel.py`, which runs each candidate in a separate process with four intra-op threads, while `train_eval.py` runs single-process. Deterministic operations are enabled in both, but the thread count changes the order of parallel reductions — the same effect Section 5.1 already measures at up to 2.7 points.

Before submission, either re-run the sweep through `scripts/sweep_architectures.py` single-process so the two figures are produced identically, or state in the text above that the sweep column was produced by a different runner and is comparable within itself but not against Table 5.3. Then delete this box.

No candidate is distinguishable from any other. The means span 0.944 to 0.972, but with only eight folds the standard error on each one is between 0.5 and 1.7 percentage points, and every confidence interval in the table overlaps every other. Even the widest separation in the set, between the GRU variant and the smallest one, is 2.8 points against a standard error of 1.7 on the difference.

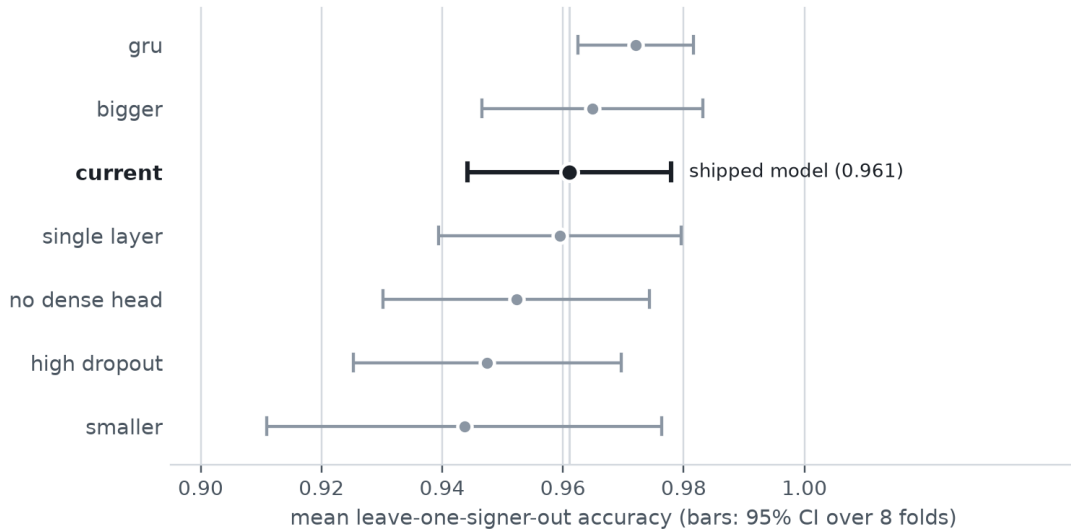


Figure 5.5: Mean leave-one-signer-out accuracy for each architecture variant, with 95% confidence intervals over the eight folds. The vertical rule marks the shipped model’s mean; every interval crosses it.

The comparison that matters more is against the spread within each candidate. The fold-to-fold standard deviation of a single architecture ranges from 0.014 to 0.047, which is larger than every difference between architectures in the table. Which signer is held out affects the result more than which architecture is trained. That is the substantive finding of this section, and Section 5.14 returns to it.

Three conclusions do follow from the table. Capacity is not the bottleneck: the largest variant carries 535,559 parameters, 2.8 times the shipped model, and gains four tenths of a point for them. A dropout rate of 0.3 is the right setting, since raising it to 0.5 is nominally worse and needs 50 epochs to converge instead of 42. And the smallest variant is the only one that looks genuinely handicapped — not for its mean, which is within noise of the rest, but for a fold-to-fold standard deviation of 0.047, more than three times the GRU variant’s, and the slowest convergence in the set. A network of 32 and 16 units is too small to behave consistently from one held-out signer to the next.

Two properties of the comparison limit what can be read from it. Each candidate was trained once per fold from a single seed, so run-to-run variation from weight initialisation is not measured and could account for differences of this size by itself. And eight folds cannot resolve differences of one or two points whatever statistical test is applied to them; that is a property of the design and not of the analysis.

The shipped architecture is therefore retained — not because it measurably outperforms the alternatives, but because no alternative measurably outperforms it, and it sits at the smaller end of the range over which accuracy has already saturated. The GRU variant is the one worth revisiting should the deployment target ever impose a size or latency budget this model misses: its accuracy advantage is within noise, but its lower fold-to-fold variance, faster convergence, and 24% smaller parameter count do not depend on that difference being real.

5.9 Why the Accuracy Is High

An accuracy near 96% on a first attempt at an under-resourced sign language invites suspicion, and it should. Five properties of the task account for it, and three of them are forms of optimism that a reader should discount.

The legitimate reasons come first. The problem is a seven-way choice, and the six phrases were drawn from different semantic domains: medical, fire, crime, and basic need. Nothing constrained them to be visually distinct, but choosing by situation produced a set whose signed forms differ in handshape, location, and movement all at once. The confusion matrix reflects this directly: across the

twenty-one possible pairs of phrases, no pair is confused more than once. Second, the landmark representation discards background, lighting, and appearance before the model sees anything, and dual-anchor normalisation removes position and scale. The nuisance variation that costs a pixel-based model most of its accuracy is not present in the input. Third, the classes are exactly balanced at 120 clips each, so no part of the score comes from favouring a common class.

The three optimistic properties matter more for interpreting the number.

Every signer reproduced a single fixed reference form, established with the consultants before recording. This was the right methodological choice, since it fixes the ground truth for each class, but it also means the dataset contains eight people imitating one performance rather than eight people signing naturally. Real signing carries regional and personal variation that this dataset largely excludes, so intra-class variance here is lower than a deployed system would meet.

Each input is one complete phrase, performed deliberately, with the user marking its start and end by tapping. The system never has to find a phrase inside continuous motion, decide where one sign ends and the next begins, or handle a signer who changes their mind mid-phrase. Segmentation is one of the harder problems in sign recognition and this design avoids it entirely.

Half the signers are the authors, who knew the phrase set better than anyone and had the most practice performing it. Their four folds average 0.9714 against 0.9387 for the other four, a gap of 3.3 points. Four folds a side is far too few to call this conclusive, and the signer 05 and signer 06 irregularities noted above sit on the weaker side of the comparison, but the direction is the one the explanation predicts and the size of it is no longer negligible. The headline figure is an accuracy on a population half of which built the system.

None of this makes 95.95% wrong. It makes it an accuracy on this task as specified: seven distinct classes, one deliberate phrase at a time, performed against a fixed reference by eight people, four of whom built the system. A wider vocabulary, natural signing, or continuous input would each lower it.

5.10 Model Export Verification

The trained model was converted to TensorFlow Lite for deployment, reducing it from 2.3 megabytes to 881 kilobytes. That is what allows it to be carried inside the application package and run on the phone, and on the fallback server it removes the need for a full TensorFlow installation.

A conversion fault would be indistinguishable from a model that simply performs slightly worse, so the export is verified against the original instead of trusted.

Both models were run on 30 real clips and their outputs compared. The largest absolute difference in any predicted probability was 1.2×10^{-7} , and the two models agreed on the predicted class for all 30 clips. The export step refuses to write its output if this check fails.

Table 5.9: TensorFlow Lite export verification

Quantity	Value
Keras model size	2.3 MB
TensorFlow Lite model size	881 KB
Clips compared	30
Largest absolute probability difference	1.2×10^{-7}
Predicted-class agreement	30 of 30

5.11 Deployment

The delivered system is a single Android application package of about 18 megabytes, of which the two MediaPipe landmark bundles account for most and the classifier for 881 kilobytes. It carries everything recognition needs and contacts nothing. The fallback server remains available as a container that omits TensorFlow entirely and runs the exported model through a lightweight runtime, which reduces that image from about three gigabytes to one.

The cost profile of a single recognition is dominated by one operation on both paths. Landmark extraction is the whole pipeline, and everything after it — the two corrections, normalisation, standardisation, and the forward pass — is a small fixed cost paid once when the user stops signing. Because frames are landmarked while the user is still signing, the dominant cost is paid concurrently with the sign rather than after it. A design that captured a whole recording and then processed it would have serialised the two, adding several seconds of silence before every result.

The difference between the two paths is where the extraction runs, and the phone is the slower of the two. That is precisely why the temporal resampling of Section 3.9.2 exists: the phone cannot be made fast enough, so the clip it produces is corrected to the timescale the model expects instead.

Table 5.10: Measured costs on the fallback server path

Stage	Cost
Landmark extraction, per frame	≈ 38 ms
Normalisation, standardisation, forward pass, per clip	≈ 15 ms
Frames retained after decimation	15.7 per second
Client send rate cap	16 frames per second
Memory per concurrent session	≈ 150 MB
Concurrent session limit	4

Concurrency on that path is bounded by memory, not processor time, since each session holds its own MediaPipe graph. The session limit sheds load with an explicit error instead of allowing the process to be terminated for exhausting memory. The on-device path has no equivalent constraint, since each phone serves exactly one user.

The application was built and installed on a physical device, and recognition works end to end with the network disabled: the user taps to begin, performs a phrase, taps to finish, and the recognised phrase appears and is spoken. The client reports its own costs as it runs, so the quantities in Table 5.11 are read from the device rather than estimated.

Table 5.11: On-device measurements, read from the client’s own telemetry on a physical device

Quantity	Value
Test device	[device model]
Sustained landmarking rate while signing	[fps]
Per-frame conversion cost	[ms]
Per-frame landmark extraction cost	[ms]
Corrections, normalisation and forward pass, per clip	[ms]
End-to-end latency, tap to spoken output	[seconds]

Screenshots of the client in each of its three result states appear in Appendix E.

5.12 Development Process

The work proceeded in eight increments, each delivering a module that could be exercised before the next began. Table 5.12 records what each produced and what it changed about the plan, since several increments revised decisions made earlier.

Table 5.12: Development increments

No.	Increment	Outcome and revisions to the plan
1	Consultation and phrase set	Signed form of each phrase fixed with Deaf experts. Face landmarks excluded after the consultants confirmed no phrase depends on expression. Phrase 6 changed to the toilet request.
2	Recorder and pilot clips	Confirmed MediaPipe produces stable landmarks for multi-sign phrases. Established the zero-fill convention and the signer-encoded filename that made later signer-independent evaluation possible.
3	Dataset collection and merge	840 clips across 8 signers, merged through Git. The none class was added here, after it became clear a six-way classifier would name a phrase for any input.
4	Preprocessing core	Sequence length derived from the measured distribution, and held in a metadata file rather than in code, which is what let it move as recordings were added. Established that normalisation must precede padding so padded frames stay exactly zero.
5	Training and evaluation	Leave-one-signer-out results. Two corrections came out of this stage: the early-stopping threshold had to be raised off zero, and deterministic operations had to be enabled before fold accuracies were reproducible. The architecture comparison of Section 5.8 was run here, once the evaluation harness existed to run it through.
6	Confidence thresholding	Decision rule fixed on the softmax over all seven classes, accepting a phrase only above threshold. The confidence threshold was lowered from 0.85 to 0.75 once the negative class was in place, since 0.85 was rejecting genuine signs. A third outcome was added here: a prediction of none refuses rather than being announced, which the two-outcome design would have spoken aloud.
7	Export and server deployment	TensorFlow Lite export with verification, inference server, and Android client. The deployment target changed from a browser to a phone at this stage, for the reasons in Section 3.9.1.
8	On-device recognition	Landmark extraction and classification moved onto the phone, removing the network dependency the previous increment had introduced. The preprocessing was ported and pinned to the Python implementation by generated fixtures, and two distortions specific to the phone — frame aspect ratio and an unattainable capture rate — were found and corrected. The server was retained as a fallback.

5.13 Behaviour as the Vocabulary Grows

The system recognises seven classes. Any extension of this work adds more, so it is worth setting out what the design predicts will happen, and which parts of it break first. No experiment in this report tests this; what follows is an argument from the structure of the method, and it identifies where the next measurements should be taken.

5.13.1 Confusion Grows Faster Than the Vocabulary

A K -class problem has $K(K - 1)/2$ pairs that can be confused. At seven classes there are 21, and the confusion matrix shows one of them carrying more than a single error. The count grows quadratically: 105 pairs at fifteen classes, 435 at thirty.

The more important effect is not the arithmetic but which pairs get added. The six phrases here were chosen by situation, not by appearance, and they happened to differ in handshape, location, and movement simultaneously. A vocabulary of thirty NSL phrases cannot be chosen that way. It must include pairs that differ in one parameter only, which is what sign-language phonology calls a minimal pair, and those are exactly the cases a 146-frame landmark sequence at 15.7 frames per second is least able to separate. Two signs distinguished by a brief change in handshape occupy a handful of frames out of 146.

The exclusion of facial landmarks becomes a limitation at this point rather than a simplification. The consultants confirmed that none of the six current phrases depends on facial expression, and that confirmation does not extend to a larger set. NSL, like other sign languages, carries grammatical information on the face, including negation and question marking. A vocabulary that includes a phrase and its negation cannot be separated by the current 225-value feature vector at all, because the information that distinguishes them is discarded before the model sees the input.

5.13.2 The Data Requirement Scales With Classes, Not Total Clips

What governs recognition quality is clips per class, not clips overall. Holding the present 120 per class, a thirty-class vocabulary needs 3,600 clips, and each new class needs its own consultation to establish its signed form before recording can begin. The consultation step, not the recording, is the binding constraint: it requires expert time from a small community.

5.13.3 The Negative Class Gets Harder

The `none` class is already the weakest at 0.862 F1, and it degrades as the vocabulary grows for a structural reason. Its job is to cover everything that is not one of the known phrases. As known phrases multiply, the boundary between the known region and everything else becomes longer and more intricate, while the number of `none` clips available to describe it stays fixed. A negative class of 120 clips describing the complement of six phrases is thin; describing the complement of thirty is thinner.

5.13.4 The Confidence Threshold Needs Recalibrating

The threshold of 0.75 requires the model to place three quarters of its probability mass on one class. Softmax spreads probability across whatever the model finds plausible, so as near-neighbours multiply the maximum probability falls even when the top prediction is correct. A fixed 0.75 would therefore reject an increasing share of correct recognitions as `low_confidence`. The threshold is a function of vocabulary size and would have to be recalibrated on validation data at each vocabulary, not carried forward.

5.13.5 Predicted Shape of the Degradation

Table 5.13 collects these effects.

Table 5.13: Predicted effect of a larger vocabulary on each component

Component	Effect of raising K	What would be needed
Phrase classifier	Confusable pairs grow as K^2 ; minimal pairs enter the set	Higher frame rate, or explicit handshape features
Feature vector	Face-dependent distinctions become unrepresentable	Add the face block, at roughly sevenfold the input width
Training data	120 clips per new class, plus one consultation each	Expert time, the scarcest input
<code>none</code> class	Describes a larger complement with the same clip count	Negative clips scaled with K
Confidence threshold	Correct predictions fall below a fixed 0.75	Recalibration per vocabulary size

The expected trajectory is therefore not a steady decline. Adding phrases that are semantically and visually distinct from the existing six should cost little, because the representation already separates such phrases well. Accuracy should fall

sharply at the point where the vocabulary first admits a minimal pair, and again where it admits a face-dependent distinction, since the second of those is not a matter of degree but of information the input does not contain.

The cheapest way to test this is to add three or four phrases chosen to be deliberately similar to existing ones and re-run the leave-one-signer-out evaluation. That measures the first of the two cliffs directly, at a cost of one consultation and a few hundred clips.

5.14 Discussion

5.14.1 What the Results Support

Under a protocol that tests only on unseen signers, the system recognises the six emergency phrases with a pooled accuracy of 95.95% and per-class F1 between 0.950 and 0.992. The landmark representation is what makes this achievable from 840 clips: a pixel-based model on the same data would have far more parameters to fit and far less invariance to the rooms and lighting the clips were recorded in. It is also what makes the result deployable, since a classifier over 225 coordinates is small enough to run on the phone that captured them.

The evaluation protocol is the part of this work most worth carrying forward. The 2.5-point gap between the random split and the signer-independent folds is a concrete measurement of how much a random split flatters a model on this kind of data, and it was obtained on a dataset small enough that the effect could easily have been dismissed as noise.

5.14.2 Eight Signers Is the Binding Limitation

Every accuracy in this chapter rests on eight people, four of whom are the authors. The standard deviation across folds is 3.40 points and the range is 9.3, which means the estimate of performance on a new signer is itself uncertain. A ninth signer with an unusual signing style could plausibly score below any fold reported here.

The architecture comparison in Section 5.8 puts this on a firmer footing than an assertion. Across seven architectures spanning 77,063 to 535,559 parameters, no variant is distinguishable from any other, and the fold-to-fold spread within each one is larger than every difference between them. What limits this system is not the model that was chosen but the number of people it has been measured on.

Four of the eight signers are also not fluent NSL users. They learned each phrase from the consultants' reference and reproduced it. The dataset therefore contains a mixture of fluent and learned performances, and the accuracy above is an accuracy on that mixture, not on fluent signing.

5.14.3 The Negative Class Is the Weak Point

The `none` class carries a recall of 0.808, and twenty-three non-signs were predicted as emergency phrases. In deployment these are the errors that produce a false announcement, which is the most serious failure mode this system can have. They are also two thirds of every error in the matrix: confusion among the six phrases has almost disappeared, and what remains is the boundary between the phrases and everything that is not one.

That single row therefore accounts for the difference between the system’s accuracy and the accuracy it could have. If every non-sign were correctly rejected, pooled accuracy would rise from 95.95% to 98.7%, and the six phrases would already be at that level.

Reducing this error is a question of what the negative class covers rather than of the threshold it is checked against. The `none` class holds 120 clips describing the complement of six phrases — an unbounded space sampled 120 times — and it is the only class whose examples are not a specific repeatable performance. Widening it with more varied non-signing motion, particularly the kind of motion these twenty-three clips resemble, is the direct way to give the classifier a better boundary to place, and it is the single most useful addition to this evaluation. How many of the twenty-three cleared the confidence threshold is not recorded in the saved metrics and would need measuring separately; that number decides whether a stricter threshold would help at all, and it should be obtained before any threshold is changed.

5.14.4 Duration Is Only Encoded Below the Sequence Length

The model has no explicit time axis and reads frame count as duration, which is why both deployment paths correct the capture rate before classifying. That reasoning holds for clips shorter than 146 frames, which is 794 of the 840. For the 39 longer clips, uniform subsampling compresses the performance to look exactly 146 frames long, so their duration information is removed. Anything longer than about 9.3 seconds of signing is therefore scale-normalised and not measured. This is an acceptable engineering choice at this vocabulary size, and it would need revisiting for longer phrases.

A related effect follows from the masking layer. A frame in which MediaPipe detected neither the pose nor either hand is all zeros, and is therefore indistinguishable from padding and skipped. A clip containing many undetected frames is consequently shorter, from the model’s point of view, than its frame count suggests.

5.14.5 What Moving On-Device Cost

Recognition runs on the phone, so the delivered system works with no network connection at all. That was the right outcome for an emergency tool, but it was not free, and the price is worth stating precisely because it is a weakness that does not show up in any accuracy figure.

The measured 95.95% was obtained on landmarks produced by MediaPipe Holistic through a webcam, preprocessed by one module. The phone produces landmarks with a different extractor, through a different lens, at a different frame rate, and preprocesses them with a port of that module rather than the module itself. Nothing in this chapter measures the accuracy of that combination. Two of the differences were large enough to find and correct — the frame aspect ratio and the capture rate, both described in Section 3.9.2 — and both were found only because they broke recognition visibly. A smaller discrepancy would not have announced itself; it would have shown up as a few more errors, indistinguishable from the model simply being imperfect.

Three things reduce that exposure without removing it. The ported preprocessing is pinned to the Python implementation by fixtures generated from it, so an arithmetic divergence fails a test. The client writes the raw landmark clip behind every attempt to storage, so any suspect result can be replayed through the Python pipeline afterwards, which is how both corrections were diagnosed. And the fallback server path remains, running the original module, so the two can be compared on the same signer at any time.

What is still missing is the measurement that would close the question: a set of clips captured through both extraction paths at once, from which the on-device accuracy could be estimated directly rather than argued for. The harness for it exists in the codebase and the recordings have not been made. It is the first item in the future work of Chapter 6.

CHAPTER 6

CONCLUSION

6.1 Conclusion

This project built a working recogniser for six Nepali Sign Language emergency phrases and delivered it as an application that displays and speaks the recognised phrase.

The signed form of every phrase was established in person with Deaf experts at the Rehabilitation Centre for the Rural Deaf and the National Federation of the Deaf Nepal before any data was recorded, so the system recognises real NSL and not an approximation of it. Eight signers then contributed 840 clips, balanced at exactly 120 per class across the six phrases and a negative class for non-signing motion. No public dataset of NSL emergency phrases existed at the start of this work, and this one is the project’s most durable output.

Representing each frame as 225 pose and hand landmarks instead of pixels is what made a dataset of this size sufficient. Normalising against the shoulder mid-point and each wrist removes the signer’s position and distance from the camera, and standardising every clip to 146 frames, the 95th percentile of the measured frame-count distribution, gives the classifier a fixed input without discarding the shape of the motion. A bidirectional LSTM of 192,007 parameters classifies the result and trains in minutes on a laptop processor.

Under leave-one-signer-out evaluation the system reaches 95.51% mean accuracy across eight folds and 95.95% pooled, with per-class F1 between 0.950 and 0.992 for the six emergency phrases. The same architecture reaches 98.41% under a random split, and reporting both is deliberate: the difference of 2.5 percentage points is a measurement of how much a random split overstates performance when clips from one person can appear on both sides of it. Published NSL recognition results are generally reported without this separation, so the lower figure here is the more comparable one to any future work that adopts the same protocol.

Because a six-way classifier would name a phrase for any input at all, the system trains a seventh, negative class on non-signing motion, reports a prediction of that class as unrecognised rather than announcing it, and requires the softmax confidence to clear a threshold before announcing anything else. For a tool whose output is a spoken emergency announcement, the ability to stay silent is a feature of the same standing as accuracy.

The trained model was exported to TensorFlow Lite, verified against the origi-

nal to a largest probability difference of 1.2×10^{-7} , and shipped inside an Android application that extracts landmarks and classifies them on the device. Recognition works end to end on a physical device with the network disabled, which is the property that matters most for a tool meant to be used in an emergency. A streaming inference server, built earlier in the project, is retained as a fallback for phones that cannot extract landmarks quickly enough.

6.2 Limitations

1. **The deployed accuracy has not been measured directly.** Every figure in this report was obtained on landmarks extracted by MediaPipe Holistic through a webcam. The phone uses a different extractor and a ported implementation of the preprocessing, so the numbers describe the method rather than the delivered application. The port is pinned to the original by generated fixtures and two on-device distortions were found and corrected, but no experiment in this report establishes what the system scores as installed. This is the most consequential limitation in this list, and the measurement that would settle it is the first item in the future work below.
2. **Eight signers, four of them the authors.** Every accuracy reported rests on eight people. The standard deviation across folds is 3.40 percentage points and the range is 9.3, so the estimate of performance on a new signer carries real uncertainty. Four of the eight are not fluent NSL users; they reproduced the consultants' reference performance. The reported accuracy is therefore an accuracy on a mixture of fluent and learned signing, and the four author folds average 3.3 points above the other four.
3. **Six phrases.** The vocabulary is fixed and small. The system cannot express anything outside it, and a user with a need not on the list has no way to convey it.
4. **The negative class is the weakest class.** Recall on none is 0.808, and twenty-three non-signs were classified as emergency phrases, each a case where the system would announce an emergency that nobody signed. These are two thirds of all errors: confusion among the six phrases themselves is now down to three isolated cases. The fix is a broader negative class rather than a stricter threshold, though how many of the twenty-three cleared the threshold has not been measured.
5. **Speech output is in English.** The client uses the device's text-to-speech

voice, and Nepali is absent from most Android devices. The proposal specified pre-recorded Nepali audio, which was not implemented.

6. **Duration is only encoded below 146 frames.** Clips longer than the sequence length are uniformly subsampled, so signing longer than about 9.3 seconds is scale-normalised rather than measured.
7. **Facial expression is ignored.** The consultants confirmed no phrase in this set depends on it, so the exclusion is safe for these six phrases and would not hold for a larger vocabulary.
8. **One sign at a time.** The system recognises a single complete phrase per interaction and does not segment continuous signing or handle more than one person in frame.
9. **Recognition speed depends on the phone.** Landmark extraction on a phone processor does not reach the capture rate of the training data, and the temporal resampling of Section 3.9.2 exists to absorb that. It restores the timescale but not the motion detail a low frame rate never sampled, so a slower device gives the model a coarser version of the same sign.

6.3 Future Work

1. **Measure the on-device path against the recorded one.** Inference already runs on the phone, but with a different landmark extractor than the dataset was recorded with, so no number in this report describes the installed system directly. The comparison that would fix this is implemented in the codebase and has not been run: it requires recording roughly 35 clips through both extraction paths at once, which isolates the landmark source from the signer, the framing and the session because the frames are the same. That measurement is the correct next step, since it either confirms the deployed accuracy or identifies what re-recording would cost.
2. **Broaden the negative class.** Recording more varied non-signing motion, targeted at the kind of motion the twenty-three misclassified clips resemble, is the direct route to reducing false announcements, and it addresses two thirds of the errors the system currently makes.
3. **Recruit more signers, prioritising fluent ones.** The clearest route to a more trustworthy accuracy figure is more people, and specifically more fluent NSL users. Doubling the signer count would narrow the confidence

in the leave-one-signer-out estimate more than any change to the model. Section 5.8 supports that ordering with a measurement rather than an argument: seven architectures spanning a sevenfold range of parameter counts differ from one another by less than the spread between signers, so architecture search has little left to give while signer coverage has a great deal.

4. **Record Nepali audio.** The phrase set is fixed at seven, so seven recordings by a fluent speaker would replace the English text-to-speech entirely.
5. **Extend the vocabulary.** The pipeline is not specific to these six phrases. Adding a phrase requires the consultation step, recordings, and a retrain, with no change to the preprocessing or the architecture.
6. **Handle continuous signing.** Recognising a phrase within a stream, with no taps to mark its boundaries, would remove the need for the user to mark the boundaries and is the main step from a proof of concept toward something usable without instruction.
7. **Even out the recordings per signer.** Signer 05 contributed no negative clips and signer 06 twice the usual number, which makes two of the eight folds harder to compare against the rest. Bringing every signer to the same per-class count would cost a short recording session and would remove the one caveat that currently has to be attached to Table 5.3.
8. **Measure how often a false announcement clears the threshold.** The confidence carried by each of the twenty-three misclassified negatives is not recorded in the saved metrics. That number decides whether the threshold has anything left to give, and obtaining it needs no new data, only an extra column in the evaluation output.

REFERENCES

- [1] K. Acharya and D. Sharma, *Nepali Sign Language Dictionary*. Kathmandu, Nepal: Nepal National Federation of the Deaf & Hard of Hearing, 2003.
- [2] J. Sunuwar, S. Borah, and A. Kharga, “NSL23 dataset for alphabets of Nepali sign language,” *Data in Brief*, vol. 52, p. 110080, 2024.
- [3] Nepali Times, “The eyes hear and the hands speak,” <https://nepalitimes.com/here-now/the-eyes-hear-and-the-hands-speak>, 2022, accessed: 2026-06-04.
- [4] Humanity & Inclusion, “Sign language: a mother tongue to Deaf children,” <https://www.hi-us.org/en/news/sign-language-a-mother-tongue-to-deaf-children>, 2023, accessed: 2026-06-04.
- [5] D. Mali, R. Mali, S. Sipai, and S. P. Pandey, “Nepali sign language translation using convolutional neural network,” in *1st KEC Conference Proceedings*, vol. 1, 2018.
- [6] G. Shrestha, N. Thing, P. Dhakal, P. Dahal, P. Karki, and M. K. Guragai, “Nepali sign language letter detection and finger spelling using MediaPipe and CNN,” in *Proceedings of the 15th IOE Graduate Conference*, vol. 15, 2024, pp. 166–170.
- [7] B. Poudel *et al.*, “Nepali sign language characters recognition: Dataset development and deep learning approaches,” *arXiv preprint arXiv:2510.11243*, 2025.
- [8] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe Hands: On-device real-time hand tracking,” *arXiv preprint arXiv:2006.10214*, 2020.
- [9] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [10] A. Graves and J. Schmidhuber, “Framewise phoneme classification with bidirectional LSTM and other neural network architectures,” *Neural Networks*, vol. 18, no. 5–6, pp. 602–610, 2005.
- [11] S. Masood, H. C. Thuwal, and A. Srivastava, “American sign language character recognition using convolution neural network,” in *Smart Computing and Informatics*. Springer, 2018, pp. 403–412.

- [12] S. Masood, A. Srivastava, H. C. Thuwal, and M. Ahmad, “Real-time sign language gesture (word) recognition from video sequences using CNN and RNN,” in *Intelligent Engineering Informatics*. Springer, 2018, pp. 623–632.
- [13] S. Ligal and D. R. Baral, “Nepali sign language gesture recognition using deep learning,” in *Proceedings of the 12th IOE Graduate Conference*, vol. 12, 2022.
- [14] K. Goyal and G. Velmathi, “Indian sign language recognition using MediaPipe Holistic,” in *Proceedings of the International Conference (VIT Chennai)*, 2023.
- [15] Google AI, “MediaPipe Holistic: Simultaneous face, hand and pose prediction, on device,” <https://research.google/blog/mediapipe-holistic-simultaneous-face-hand-and-pose-prediction-on-device/>, 2020, accessed: 2026-07-12.
- [16] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boulton, “Toward open set recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013.
- [17] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, vol. 2, pp. 1137–1143, 1995.
- [18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [20] Google, “LiteRT (formerly TensorFlow Lite): On-device machine learning,” <https://ai.google.dev/edge/litert>, 2024, accessed: 2026-08-05.

APPENDIX A

Field Consultation with NSL Experts

The signed form of each of the six emergency phrases was established in person with Deaf experts before any data collection began, as described in Section 3.2. Consultations were held at the Rehabilitation Centre for the Rural Deaf in Bhaktapur and at the National Federation of the Deaf Nepal in Ranibari Marga, Samakhushi, Kathmandu. The images below are reproduced with the consent of those photographed.



Figure A.1: The project team with the NSL consultant following a gloss consultation session



Figure A.2: Reviewing a recorded demonstration with the consultant to confirm the reference form of a phrase



Figure A.3: An NSL expert demonstrating the signed form of a phrase for reference recording

APPENDIX B

Data Collection Tool

Clips were recorded with a purpose-built tool that runs MediaPipe Holistic on a live webcam feed and saves the 225-value landmark vector for each frame. The operator selects a signer identity and a phrase, then starts and stops each recording explicitly. Landmark overlays are drawn live, so a failed detection is visible during recording instead of afterwards.

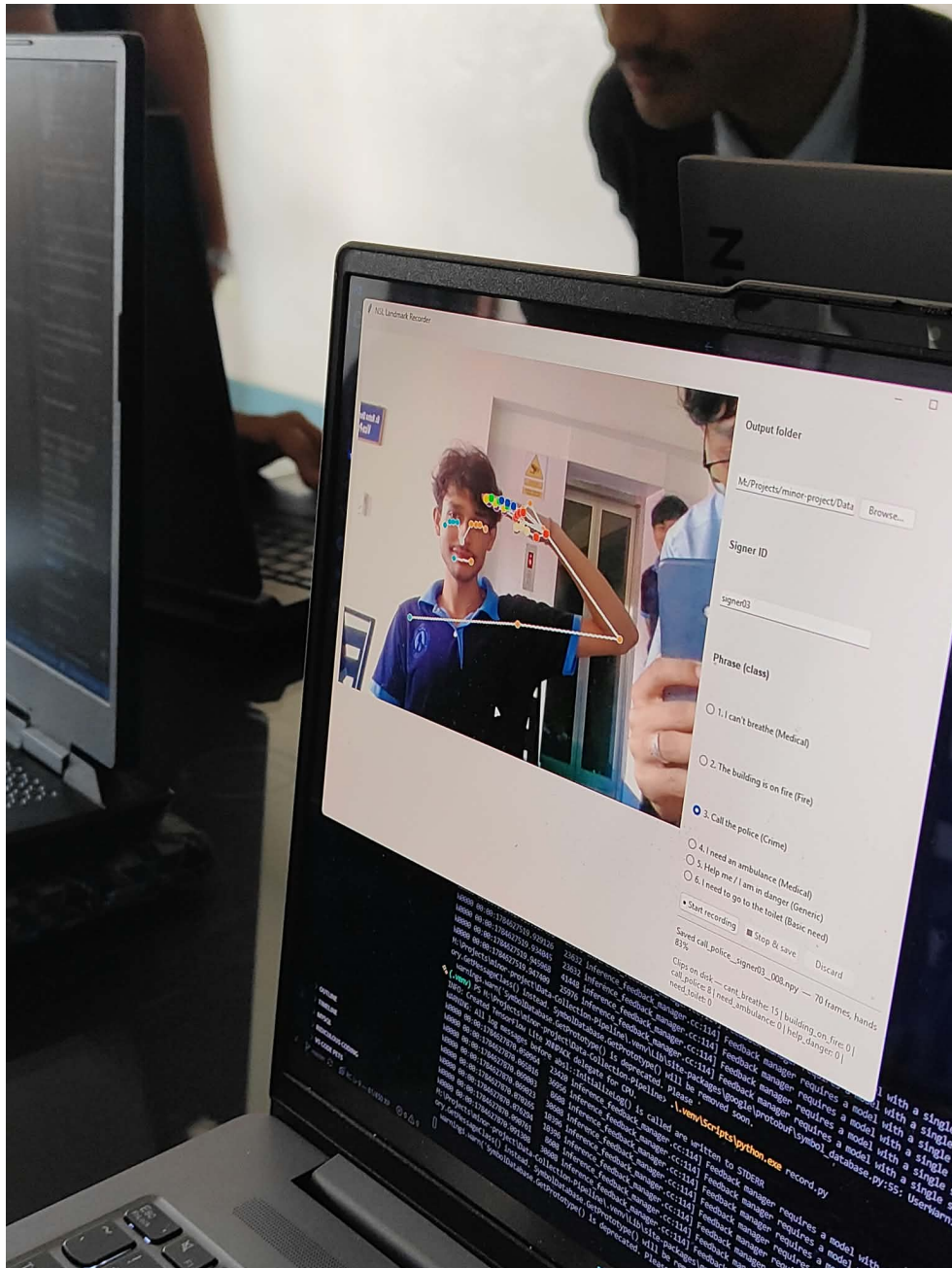


Figure B.1: A signer recording the *Call the police* phrase, showing live pose and hand landmark overlays alongside the signer identity and phrase selection

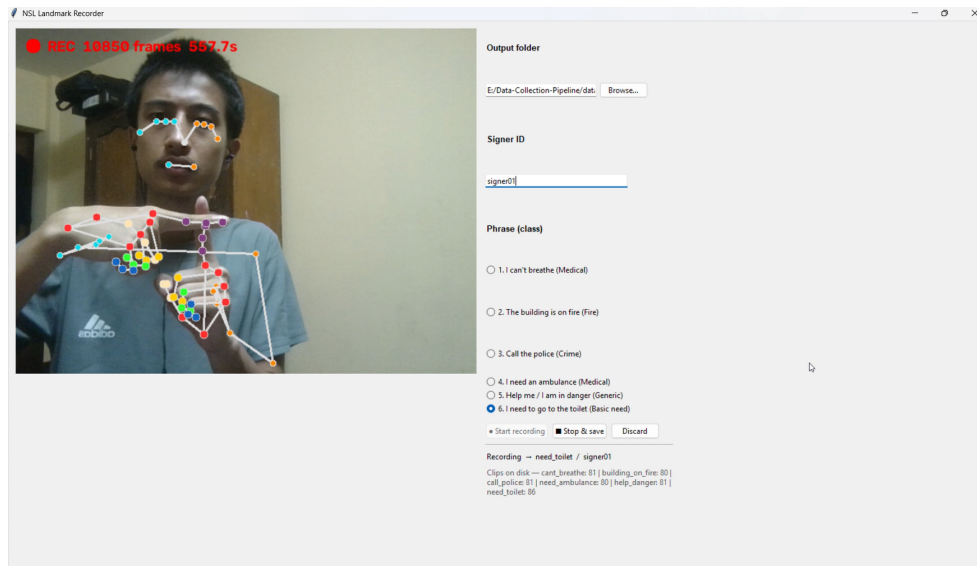


Figure B.2: The recorder during an active recording, showing hand landmarks in detail and the running per-class clip counts on disk

Dataset Organisation

The eight signers recorded independently on separate machines. Contributions were merged through a shared Git repository instead of by hand, which gave a reviewable history of what each signer added and caught duplicate and misnamed clips before they entered the combined dataset.

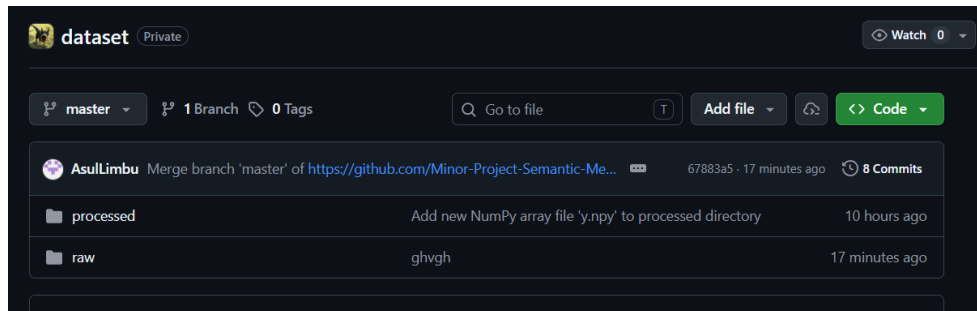


Figure C.1: The shared dataset repository, with raw and processed directories tracked separately and a commit history showing contributions merged from several signers

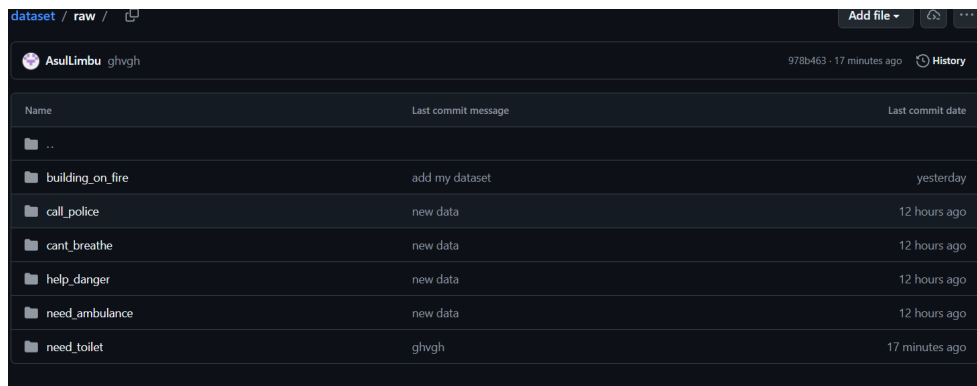


Figure C.2: The raw directory after merging, with one subfolder per phrase class holding the contributions of all signers



Figure C.3: Local directory layout: raw clips organised by phrase class, alongside the processed output directory

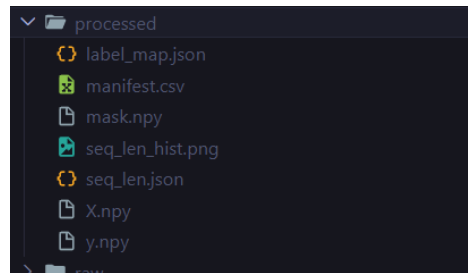


Figure C.4: Contents of the processed directory: the compiled tensors alongside the manifest and the sequence-length metadata

APPENDIX D

Sample Landmark Data

This appendix traces one clip through the preprocessing pipeline, to make the numeric effect of each step concrete. The clip is `call_police__signer01__001.npy`, stored as a $(42, 225)$ array of 32-bit floats: 42 recorded frames, each holding one 225-value landmark vector.

D.1 Feature Vector Layout

Table D.1: Layout of the 225-value feature vector

Block	Indices	Values	Content
Pose	0 to 98	99	33 landmarks $\times (x, y, z)$
Left hand	99 to 161	63	21 landmarks $\times (x, y, z)$
Right hand	162 to 224	63	21 landmarks $\times (x, y, z)$
Total		225	

D.2 Raw Values

The first five pose landmarks of the clip’s first frame, in the normalised image coordinates MediaPipe reports:

Table D.2: Raw pose landmarks 0 to 4, first frame

Landmark	x	y	z
0	0.3683	0.3315	−1.0291
1	0.3924	0.2687	−0.9459
2	0.4069	0.2707	−0.9461
3	0.4187	0.2732	−0.9460
4	0.3420	0.2707	−0.9640

Both hand blocks of this frame are entirely zero. The signer’s hands had not yet entered the frame when recording began, MediaPipe reported no hand landmarks, and the recorder filled all 126 values with zeros instead of leaving them undefined.

D.3 After Normalisation

The same five landmarks after dual-anchor normalisation, now expressed relative to the shoulder midpoint and scaled by shoulder width:

Table D.3: Normalised pose landmarks 0 to 4, first frame

Landmark	x	y	z
0	-0.0054	-0.7148	-1.6152
1	0.0463	-0.8491	-1.4370
2	0.0772	-0.8449	-1.4374
3	0.1025	-0.8396	-1.4374
4	-0.0617	-0.8449	-1.4758

The x coordinates now sit around zero instead of around 0.37, because the origin has moved to the midpoint of the shoulders. The zero hand blocks remain exactly zero, since both normalisation formulas map a zero input to a zero output.

D.4 After Length Standardisation

The clip has 42 frames, fewer than the sequence length of 146, so it is zero-padded at the end. The result is a (146, 225) array in which the first 42 frames are real and the remaining 104 are zero, with a boolean mask recording which is which. The final frame of the standardised clip is entirely zero, confirming that padding stays distinguishable from signal and that the masking layer will skip it.

APPENDIX E

Client Interface

The Android client presents one of three outcomes for every recognition attempt: an accepted phrase, which is displayed and spoken; an unrecognised result, when the classifier identifies the input as not being one of the six phrases; or a low-confidence result, when the top class is a phrase but the model will not commit to it. Only the first produces speech. A row of indicators below the camera preview reports what the recogniser observed for each frame — whether pose and hands were detected, the sustained frame rate, the per-frame conversion and extraction costs, and whether any frames were dropped — so a failure during use can be attributed instead of guessed at.

[Insert client screenshots here: the accepted, unrecognised and low-confidence result states, and the telemetry indicator row during an active recording. The server configuration screen is optional, since it is reachable only after switching out of the default on-device mode.]

APPENDIX F

Server Interface

The fallback inference server exposes the endpoints in Table F.1. The streaming endpoint is the path the client uses when on-device recognition is switched off; the two prediction endpoints exist so that the served decision can be checked against the offline pipeline without needing a phone at all, which is what keeps this path useful as a reference even when nothing uses it in production.

Table F.1: Server endpoints

Method	Path	Key	Purpose
GET	/health	no	Backend, class list, thresholds, target frame rate
GET	/classes	yes	Class keys and display labels
POST	/predict/landmarks	yes	A clip supplied as JSON, for verification
POST	/predict/npv	yes	The same clip as raw array bytes
WS	/ws/stream	yes	Live frames to a decision, the path the client uses

The health endpoint is deliberately unauthenticated so that container health checks and the client’s reachability test keep working when a key is configured. Every other endpoint requires the key, compared with a constant-time function so that a wrong key cannot be narrowed down by timing the response.

Table F.2: Messages on the streaming endpoint

Direction	Message	Meaning
Server	ready	Session established; carries the target frame rate and class list
Client	binary frame	One JPEG frame
Server	ack	Whether the frame was kept, the running frame count, and whether pose and hands were detected
Client	done	Signing has finished; produce a decision
Server	result	Status, phrase, confidence, and the three most probable classes
Client	reset	Discard the current clip and start again
Client	ping	Keepalive, sent while idle between signs

The on-device path carries no protocol, but it produces the same result structure — status, phrase, confidence, and the top three classes — so the presentation logic

in the client is identical whichever path produced the decision. Switching between them changes where the computation happens and nothing the user sees.